Red Hat OpenShift AI Self-Managed 3.0

Managing OpenShift AI

Cluster administrator tasks for managing OpenShift AI

Last Updated: 2025-12-01

Red Hat OpenShift AI Self-Managed 3.0 Managing OpenShift AI

Cluster administrator tasks for managing OpenShift AI

Legal Notice

Abstract

As an OpenShift cluster administrator, manage OpenShift AI users and groups, the dashboard interface and applications, deployment resources, accelerators, distributed workloads, and data backup.

Table of Contents

PREFACE

As an OpenShift cluster administrator, you can manage the following Red Hat OpenShift AI resources:

- Users and groups
- Custom workbench images
- Applications that show in the dashboard
- Custom deployment resources that are related to the Red Hat OpenShift AI Operator, for example, CPU and memory limits and requests
- Accelerators
- Workload resources with Kueue
- Distributed workloads
- Configure external OIDC identity providers
- Data backup
- Monitoring and observability
- Logs and audit records

CHAPTER 1. MANAGING USERS AND GROUPS

Users with cluster administrator access to OpenShift can add, modify, and remove user permissions for Red Hat OpenShift AI.

1.1. OVERVIEW OF USER TYPES AND PERMISSIONS

Table 1 describes the Red Hat OpenShift AI user types.

Table 1.1. User types

User Type	Permissions
Users	Machine learning operations (MLOps) engineers and data scientists can access and use individual components of Red Hat OpenShift AI, such as workbenches and AI pipelines. See also Accessing the OpenShift AI dashboard
Administrators	In addition to the actions permitted to users, administrators can perform these actions: • Configure Red Hat OpenShift AI settings. • Access and manage workbenches. • Access and manage pipeline applications for any project.

By default, all OpenShift users have access to Red Hat OpenShift AI. In addition, users in the OpenShift administrator group (**cluster admins**), automatically have administrator access in OpenShift AI.

Optionally, if you want to restrict access to your OpenShift AI deployment to specific users or groups, you can create user groups for users and administrators.

If you decide to restrict access, and you already have groups defined in your configured identity provider, you can add these groups to your OpenShift AI deployment. If you decide to use groups without adding these groups from an identity provider, you must create the groups in OpenShift and then add users to them.

There are some operations relevant to OpenShift AI that require the **cluster-admin** role. Those operations include:

- Adding users to the OpenShift AI user and administrator groups, if you are using groups.
- Removing users from the OpenShift AI user and administrator groups, if you are using groups.
- Managing custom environment and storage configuration for users in OpenShift, such as Jupyter notebook resources, ConfigMaps, and persistent volume claims (PVCs).

IMPORTANT

Although users of OpenShift AI and its components are authenticated through OpenShift, session management is separate from authentication. This means that logging out of OpenShift or OpenShift AI does not affect a logged in Jupyter session running on those platforms. This means that when a user's permissions change, that user must log out of all current sessions in order for the changes to take effect.

Additional resources

See also OpenShift Container Platform Authentication and authorization .

1.2. VIEWING OPENSHIFT AI USERS

If you have defined OpenShift AI user groups, you can view the users that belong to these groups.

Prerequisites

- The Red Hat OpenShift AI user group, administrator group, or both exist.
- You have the **cluster-admin** role in OpenShift.
- You have configured a supported identity provider for OpenShift.

Procedure

1. In the OpenShift web console, click **User Management → Groups**.
2. Click the name of the group containing the users that you want to view.
   - For administrative users, click the name of your administrator group. for example, **rhods-admins**.
   - For normal users, click the name of your user group, for example, **rhods-users**.
     The **Group details** page for the group is displayed.

Verification

- In the **Users** section for the relevant group, you can view the users who have permission to access Red Hat OpenShift AI.

1.3. ADDING USERS TO OPENSHIFT AI USER GROUPS

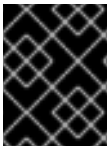
By default, all OpenShift users have access to Red Hat OpenShift AI.

Optionally, you can restrict user access to your OpenShift AI instance by defining user groups. You must grant users permission to access Red Hat OpenShift AI by adding user accounts to the Red Hat OpenShift AI user group, administrator group, or both. You can either use the default group name, or specify a group name that already exists in your identity provider.

The **user group** provides the user with access to product components in the Red Hat OpenShift AI dashboard, such as AI pipelines, and associated services, such as Jupyter. By default, users in the **user group** have access to AI pipeline applications within projects that they created.

The **administrator group** provides the user with access to developer and administrator functions in the Red Hat OpenShift AI dashboard, such as AI pipelines, and associated services, such as Jupyter. Users in the **administrator group** can configure AI pipeline applications in the OpenShift AI dashboard for any project.

If you restrict access by using user groups, users that are not in the OpenShift AI user group or administrator group cannot view the dashboard and use associated services, such as Jupyter. They are also unable to access the **Cluster settings** page.



IMPORTANT

If you are using LDAP as your identity provider, you need to configure LDAP syncing to OpenShift. For more information, see Syncing LDAP groups.

Follow the steps in this section to add users to your OpenShift AI administrator and user groups.

Note: You can add users in OpenShift AI but you must manage the user lists in the OpenShift web console.

Prerequisites

- You have configured a supported identity provider for OpenShift.
- You are assigned the **cluster-admin** role in OpenShift.
- You have defined an administrator group and user group for OpenShift AI.

Procedure

1. In the OpenShift web console, click **User Management → Groups**.
2. Click the name of the group you want to add users to.
   - For administrative users, click the administrator group, for example, **rhods-admins**.
   - For normal users, click the user group, for example, **rhods-users**.
     The **Group details** page for that group opens.
3. Click **Actions → Add Users**.
   The **Add Users** dialog opens.
4. In the **Users** field, enter the relevant user name to add to the group.
5. Click **Save**.

Verification

- Click the **Details** tab for each group and confirm that the **Users** section contains the user names that you added.

1.4. SELECTING OPENSHIFT AI ADMINISTRATOR AND USER GROUPS

By default, all users authenticated in OpenShift can access OpenShift AI.

Also by default, users with **cluster-admin** permissions are OpenShift AI administrators. A **cluster admin** is a superuser that can perform any action in any project in the OpenShift cluster. When bound to a user with a local binding, they have full control over quota and every action on every resource in the project.

After a **cluster admin** user defines additional administrator and user groups in OpenShift, you can add those groups to OpenShift AI by selecting them in the OpenShift AI dashboard.

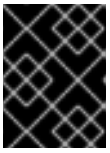
Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.

- The groups that you want to select as administrator and user groups for OpenShift AI already exist in OpenShift. For more information, see Managing users and groups.

Procedure

1. From the OpenShift AI dashboard, click **Settings → User management**.
2. Select your OpenShift AI administrator groups: Under **Red Hat OpenShift AI administrator groups**, click the text box and select an OpenShift group. Repeat this process to define multiple administrator groups.
3. Select your OpenShift AI user groups: Under **Red Hat OpenShift AI user groups**, click the text box and select an OpenShift group. Repeat this process to define multiple user groups.



IMPORTANT

The **system:authenticated** setting allows all users authenticated in OpenShift to access OpenShift AI.

4. Click **Save changes**.

Verification

- Administrator users can successfully log in to OpenShift AI and have access to the **Settings** navigation menu.
- Non-administrator users can successfully log in to OpenShift AI. They can also access and use individual components, such as projects and workbenches.

1.5. DELETING USERS

1.5.1. About deleting users and their resources

If you have administrator access to OpenShift, you can revoke a user's access to workbenches and delete the user's resources from Red Hat OpenShift AI. Before you delete a user from OpenShift AI, it is good practice to back up the data on your persistent volume claims (PVCs).

Deleting a user and the user's resources involves the following tasks:

- Stop workbenches owned by the user.
- Revoke user access to workbenches.
- Remove the user from the allowed group in your OpenShift identity provider.
- After you delete a user, delete their associated configuration files from OpenShift.

1.5.2. Stopping basic workbenches owned by other users

OpenShift AI administrators can stop basic workbenches that are owned by other users to reduce resource consumption on the cluster, or as part of removing a user and their resources from the cluster.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have launched the **Start basic workbench** application, as described in Starting a basic workbench.
- The workbench that you want to stop is running.

Procedure

1. On the page that opens when you launch a basic workbench, click the **Administration** tab.
2. Stop one or more servers.
   - If you want to stop one or more specific servers, perform the following actions:
     i. In the **Users** section, locate the user that the workbench belongs to.
     ii. To stop the workbench, perform one of the following actions:
       - Click the action menu (⋮) beside the relevant user and select **Stop server**.
       - Click **View server** beside the relevant user and then click **Stop workbench**. The **Stop server** dialog box opens.

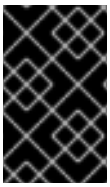
     iii. Click **Stop server**.
   - If you want to stop all workbenches, perform the following actions:
     i. Click the **Stop all workbenches** button.
     ii. Click **OK** to confirm stopping all servers.

Verification

- The **Stop server** link beside each server changes to a **Start workbench** link when the workbench has stopped.

1.5.3. Revoking user access to basic workbenches

You can revoke a user's access to basic workbenches by removing the user from the OpenShift AI user groups that define access to OpenShift AI. When you remove a user from the user groups, the user is prevented from accessing the OpenShift AI dashboard and from using associated services that consume resources in your cluster.



IMPORTANT

Follow these steps only if you have implemented OpenShift AI user groups to restrict access to OpenShift AI. To completely remove a user from OpenShift AI, you must remove them from the allowed group in your OpenShift identity provider.

Prerequisites

- You have stopped any workbenches owned by the user you want to delete.
- You are assigned the **cluster-admin** role in OpenShift.

- You are using OpenShift AI user groups, and the user is part of the user group, administrator group, or both.

Procedure

1. In the OpenShift web console, click **User Management → Groups**.
2. Click the name of the group that you want to remove the user from.
   - For administrative users, click the name of your administrator group, for example, **rhods-admins**.
   - For non-administrator users, click the name of your user group, for example, **rhods-users**.

   The **Group details** page for the group is displayed.
3. In the **Users** section on the **Details** tab, locate the user that you want to remove.
4. Click the action menu (⋮) beside the user that you want to remove and click **Remove user**.

Verification

- In the **Users** section on the **Details** tab of the **Group details** page, confirm that the user that you removed is not visible. In **Workloads → Pods**, select the default workbench project (**rhods-notebooks** or your custom workbench namespace), and ensure that there is no workbench pod for this user. If you see a pod named **jupyter-nb-<username>-*** for the user that you have removed, delete that pod to ensure that the deleted user is not consuming resources on the cluster.
- In the OpenShift AI dashboard, check the list of projects. Delete any projects that belong to the user.

1.5.4. Backing up storage data

It is a best practice to back up the data on your persistent volume claims (PVCs) regularly.

Backing up your data is particularly important before you delete a user and before you uninstall OpenShift AI, as all PVCs are deleted when OpenShift AI is uninstalled.

For more information about backing up PVCs for your cluster platform, see OADP Application backup and restore in the OpenShift Container Platform documentation.

Additional resources

- Understanding persistent storage

1.5.5. Cleaning up after deleting users

After you remove a user's access to Red Hat OpenShift AI, you must also delete the configuration files for the user from OpenShift. Red Hat recommends that you back up the user's data before removing their configuration files.

Prerequisites

- (Optional) If you want to completely remove the user's access to OpenShift AI, you have removed their credentials from your identity provider.
- You have backed up the user's storage data.
- You have logged in to the OpenShift web console as a user with the **cluster-admin** role.

Procedure

1. Delete the user's persistent volume claim (PVC).
   a. Click **Storage → PersistentVolumeClaims**.
   b. If it is not already selected, select the default workbench project (**rhods-notebooks** or your custom workbench namespace) from the project list.
   c. Locate the **jupyter-nb-<username>** PVC.
   Replace **<username>** with the relevant user name.
   d. Click the action menu (⋮) and select **Delete PersistentVolumeClaim** from the list.
   The **Delete PersistentVolumeClaim** dialog opens.
   e. Inspect the dialog and confirm that you are deleting the correct PVC.
   f. Click **Delete**.
2. Delete the user's ConfigMap.
   a. Click **Workloads → ConfigMaps**.
   b. If it is not already selected, select the default workbench project (**rhods-notebooks** or your custom workbench namespace) from the project list.
   c. Locate the **jupyterhub-singleuser-profile-<username>** ConfigMap.
   Replace **<username>** with the relevant user name.
   d. Click the action menu (⋮) and select **Delete ConfigMap** from the list.
   The **Delete ConfigMap** dialog opens.
   e. Inspect the dialog and confirm that you are deleting the correct ConfigMap.
   f. Click **Delete**.

Verification

- The user cannot access OpenShift AI and sees an "Access permission needed" message if they try.
- The user's single-user profile, persistent volume claim (PVC), and ConfigMap are not visible in OpenShift.

CHAPTER 2. CREATING CUSTOM WORKBENCH IMAGES

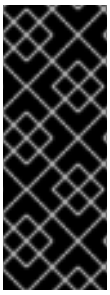
Red Hat OpenShift AI includes a selection of default workbench images that a data scientist can select when they create or edit a workbench.

In addition, you can import a custom workbench image, for example, if you want to add libraries that data scientists often use, or if your data scientists require a specific version of a library that is different from the version provided in a default image. Custom workbench images are also useful if your data scientists require operating system packages or applications because they cannot install them directly in their running environment (data scientist users do not have root access, which is needed for those operations).

A custom workbench image is simply a container image. You build one as you would build any standard container image, by using a Containerfile (or Dockerfile). You start from an existing image (the **FROM** instruction), and then add your required elements.

You have the following options for creating a custom workbench image:

- Start from one of the default images, as described in Creating a custom image from a default OpenShift AI image.
- Create your own image by following the guidelines for making it compatible with OpenShift AI, as described in Creating a custom image from your own image .



IMPORTANT

Red Hat supports adding custom workbench images to your deployment of OpenShift AI, ensuring that they are available for selection when creating a workbench. However, Red Hat does not support the contents of your custom workbench image. That is, if your custom workbench image is available for selection during workbench creation, but does not create a usable workbench, Red Hat does not provide support to fix your custom workbench image.

Additional resources

For a list of the OpenShift AI default workbench images and their preinstalled packages, see Supported Configurations for 3.x.

For more information about creating images, see the following resources:

- Red Hat OpenShift Container Platform - Creating Images
- Red Hat OpenShift Service on AWS - Creating images
- Red Hat OpenShift Dedicated - Dockerfile reference documentation

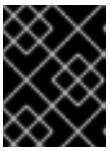
2.1. CREATING A CUSTOM IMAGE FROM A DEFAULT OPENSHIFT AI IMAGE

After Red Hat OpenShift AI is installed on a cluster, you can find the default workbench images in the OpenShift console, under **Builds → ImageStreams** for the **redhat-ods-applications** project.

You can create a custom image by adding OS packages or applications to a default OpenShift AI image.

Prerequisites

- You know which default image you want to use as the base for your custom image.



IMPORTANT

If you want to create a custom Elyra-compatible image, the base image must be an OpenShift AI image that contains the Elyra extension.

See Supported Configurations for 3.x for a list of the OpenShift AI default workbench images and their preinstalled packages.

- You have **cluster-admin** access to the OpenShift console for the cluster where OpenShift AI is installed.

Procedure

1. Obtain the location of the default image that you want to use as the base for your custom image.
   a. In the OpenShift console, select **Builds → ImageStreams**.
   b. Select the **redhat-ods-applications** project.
   c. From the list of installed imagestreams, click the name of the image that you want to use as the base for your custom image. For example, click **pytorch**.
   d. On the ImageStream details page, click **YAML**.
   e. In the **spec:tags** section, find the tag for the version of the image that you want to use. The location of the original image is shown in the tag's **from:name** section, for example:

      name: 'quay.io/modh/odh-pytorch-notebook@sha256:b68e0192abf7d…'

   f. Copy this location for use in your custom image.
2. Create a standard Containerfile or Dockerfile.
3. For the **FROM** instruction, specify the base image location that you copied in Step 1, for example:
   FROM quay.io/modh/odh-pytorch-notebook@sha256:b68e0…
4. Optional: Install OS images:
   a. Switch to **USER 0** (USER 0 is required to install OS packages).
   b. Install the packages.
   c. Switch back to **USER 1001**.
   The following example creates a custom workbench image that adds Java to the default PyTorch image:

```
FROM quay.io/modh/odh-pytorch-notebook@sha256:b68e0…

USER 0

RUN INSTALL_PKGS="java-11-openjdk java-11-openjdk-devel" && \
```

```
    dnf install -y --setopt=tsflags=nodocs $INSTALL_PKGS && \
    dnf -y clean all --enablerepo=*

USER 1001
```

5. Optional: Add Python packages:
   a. Specify **USER 1001**.
   b. Copy the **requirements.txt** file.
   c. Install the packages.
   The following example installs packages from the **requirements.txt** file in the default PyTorch image:

```
FROM quay.io/modh/odh-pytorch-notebook@sha256:b68e0…

USER 1001

COPY requirements.txt ./requirements.txt

RUN pip install -r requirements.txt
```

6. Build the image file. For example, you can use **podman build** locally where the image file is located and then push the image to a registry that is accessible to OpenShift AI:

```
$ podman build -t my-registry/my-custom-image:0.0.1 .
$ podman push my-registry/my-custom-image:0.0.1
```

   Alternatively, you can leverage OpenShift's image build capabilities by using BuildConfig.

2.2. CREATING A CUSTOM IMAGE FROM YOUR OWN IMAGE

You can build your own custom image. However, you must make sure that your image is compatible with OpenShift and OpenShift AI.

Additional resources

- General Container image guidelines section in the OpenShift Container Platform Images documentation.
- Red Hat Universal Base Image: https://catalog.redhat.com/software/base-images
- Red Hat Ecosystem Catalog: https://catalog.redhat.com/

2.2.1. Basic guidelines for creating your own workbench image

The following basic guidelines provide information to consider when you build your own custom workbench image.

Designing your image to run with USER 1001

In OpenShift, your container will run with a random UID and a GID of **0**. Make sure that your image is compatible with these user and group requirements, especially if you need write access to directories. Best practice is to design your image to run with **USER 1001**.

Avoid placing artifacts in $HOME

The persistent volume attached to the workbench will be mounted on **/opt/app-root/src**. This location is also the location of **$HOME**. Therefore, do not put any files or other resources directly in **$HOME** because they are not visible after the workbench is deployed (and the persistent volume is mounted).

Specifying the API endpoint

OpenShift readiness and liveness probes will query the **/api** endpoint. For a Jupyter IDE, this is the default endpoint. For other IDEs, you must implement the **/api** endpoint.

2.2.2. Advanced guidelines for creating your own workbench image

The following guidelines provide information to consider when you build your own custom workbench image.

Minimizing image size

A workbench image uses a "layered" file system. Every time you use a COPY or a RUN command in your workbench image file, a new layer is created. Artifacts are not deleted. When you remove an artifact, for example, a file, it is "masked" in the next layer. Therefore, consider the following guidelines when you create your workbench image file.

- Avoid using the **dnf update** command.
    - If you start from an image that is constantly updated, such as **ubi9/python-39** from the Red Hat Catalog, you might not need to use the **dnf update** command. This command fetches new metadata, updates files that might not have impact, and increases the workbench image size.
    - Point to a newer version of your base image rather than performing a **dnf update** on an older version.
- Group **RUN** commands. Chain your commands by adding **&&** \ at the end of each line.
- If you must compile code (such as a library or an application) to include in your custom image, implement multi-stage builds so that you avoid including the build artifacts in your final image. That is, compile the library or application in an intermediate image and then copy the result to your final image, leaving behind build artifacts that you do not want included.

Setting access to files and directories

- Set the ownership of files and folders to **1001:0** (user "default", group "0"), for example:

    ```
    COPY --chown=1001:0 os-packages.txt ./
    ```

    On OpenShift, every container is in a standard namespace (unless you modify security). The container runs with a user that has a random user ID (uid) and with a group ID (gid) of **0**. Therefore, all folders that you want to write to - and all the files you want to (temporarily) modify - in your image must be accessible by the user that has the random user ID (uid). Alternatively, you can set access to any user, as shown in the following example:


```
COPY --chmod=775 os-packages.txt ./
```

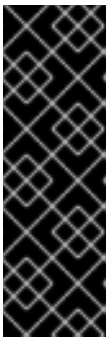
- Build your image with **/opt/app-root/src** as the default location for the data that you want persisted, for example:

```
WORKDIR /opt/app-root/src
```

When a user launches a workbench from the OpenShift AI **Applications → Enabled** page, the personal volume of the user is mounted in the user's HOME directory (**/opt/app-root/src**). Because this location is not configurable, when you build your custom image, you must specify this default location for persisted data.

- Fix permissions to support PIP (the package manager for Python packages) in OpenShift environments. Add the following command to your custom image (if needed, change **python3.11** to the Python version that you are using):

```
chmod -R g+w /opt/app-root/lib/python3.11/site-packages && \
  fix-permissions /opt/app-root -P
```



IMPORTANT

Your workbench image needs to serve all of its content from the base path **/${NB_PREFIX}** because the routing, handled by the Gateway API, is path-based and uses the same value as the environment variable **NB_PREFIX**.

The **NB_PREFIX** environment variable is injected in the workbench at runtime. Any call from a browser to a different path, for example **/index.html**, **/api/my-endpoint**, or simply /, will not be routed to the workbench container.

- A service within your workbench image must answer at **${NB_PREFIX}/api**, otherwise the OpenShift liveness/readiness probes fail and delete the pod for the workbench image. The **NB_PREFIX** environment variable specifies the URL path where the container is expected to be listening.

The following is an example of an Nginx configuration:

```
location = ${NB_PREFIX}/api {
 return 302  /healthz;
 access_log  off;
}
```

- For idle culling to work, the **${NB_PREFIX}/api/kernels** URL must return a specifically-formatted JSON payload, as shown in the following example:
The following is an example of an Nginx configuration:

```
location = ${NB_PREFIX}/api/kernels {
 return 302 $custom_scheme://$http_host/api/kernels/;
 access_log  off;
}

location ${NB_PREFIX}/api/kernels/ {
 return 302 $custom_scheme://$http_host/api/kernels/;
 access_log  off;
```

```
}

location /api/kernels/ {
  index access.cgi;
  fastcgi_index access.cgi;
  gzip  off;
  access_log off;
 }
```

The returned JSON payload should be:

```
{"id":"rstudio","name":"rstudio","last_activity":(time in ISO8601
format),"execution_state":"busy","connections": 1}
```

Enabling CodeReady Builder (CRB) and Extra Packages for Enterprise Linux (EPEL)

CRB and EPEL are repositories that provide packages which are absent from a standard Red Hat Enterprise Linux (RHEL) or Universal Base Image (UBI) installation. They are useful and required for installing some software, for example, RStudio.

On UBI9 images, CRB is enabled by default. To enable EPEL on UBI9-based images, run the following command:

```
RUN yum install -y https://download.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

To enable CRB and EPEL on Centos Stream 9-based images, run the following command:

```
RUN yum install -y yum-utils && \
  yum-config-manager --enable crb && \
  yum install -y https://download.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

2.3. ENABLING CUSTOM IMAGES IN OPENSHIFT AI

All OpenShift AI administrators can import custom workbench images, by default, by selecting the **Settings → Environment setup → Workbench images** navigation option in the OpenShift AI dashboard.

If the **Settings → Environment setup → Workbench images** option is not available, check the following settings, depending on which navigation element does not appear in the dashboard:

- The **Settings** menu does not appear in the OpenShift AI navigation bar.
  The visibility of the OpenShift AI dashboard **Settings** menu is determined by your user permissions. By default, the **Settings** menu is available to OpenShift AI administration users (users that are members of the **rhods-admins** group). Users with the OpenShift **cluster-admin** role are automatically added to the **rhods-admins** group and are granted administrator access in OpenShift AI.

  For more information about user permissions, see Managing users and groups.

- The **Workbench images** menu item does not appear under the **Settings** menu.
  The visibility of the **Workbench images** menu item is controlled in the dashboard configuration, by the value of the **dashboardConfig: disableBYONImageStream** option. It is set to **false** (the **Workbench images** menu item is visible) by default.

You need OpenShift AI administrator permissions to edit the dashboard configuration.

For more information about setting dashboard configuration options, see Customizing the dashboard.

2.4. IMPORTING A CUSTOM WORKBENCH IMAGE

In addition to workbench images provided and supported by Red Hat and independent software vendors (ISVs), you can import custom workbench images that cater to your project's specific requirements.

You must import it so that your OpenShift AI users (data scientists) can access it when they create a project workbench.

Red Hat supports adding custom workbench images to your deployment of OpenShift AI, ensuring that they are available for selection when creating a workbench. However, Red Hat does not support the contents of your custom workbench image. That is, if your custom workbench image is available for selection during workbench creation, but does not create a usable workbench, Red Hat does not provide support to fix your custom workbench image.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- Your custom image exists in an image registry that is accessible to OpenShift AI.
- The **Settings → Environment setup → Workbench images** dashboard navigation menu item is enabled, as described in Enabling custom workbench images in OpenShift AI .
- If you want to associate an accelerator with the custom image that you want to import, you know the accelerator's identifier - the unique string that identifies the hardware accelerator. You must also have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see Installing the Node Feature Discovery Operator and Enabling NVIDIA GPUs.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Environment setup → Workbench images**. The **Workbench images** page opens. Previously imported images are displayed. To enable or disable a previously imported image, on the row containing the relevant image, click the toggle in the **Enable** column.
2. Optional: If you want to associate an accelerator and you have not already created an accelerator profile or a hardware profile, click **Create profile** on the row containing the image and complete the relevant fields. If the image does not contain an accelerator identifier, you must manually configure one before creating an associated accelerator profile or a hardware profile.
3. Click **Import new image**. Alternatively, if no previously imported images were found, click **Import image**.
   The **Import workbench image** dialog opens.
4. In the **Image location** field, enter the URL of the repository containing the image. For example: **quay.io/my-repo/my-image:tag**, **quay.io/my-repo/my-image@sha256:xxxxxxxxxxxx**, or **docker.io/my-repo/my-image:tag**.

5. In the **Name** field, enter an appropriate name for the image.
6. Optional: In the **Description** field, enter a description for the image.
7. Optional: From the **Accelerator identifier** list, select an identifier to set its accelerator as recommended with the image. If the image contains only one accelerator identifier, the identifier name displays by default.
8. Optional: Add software to the image. After the import has completed, the software is added to the image's meta-data and displayed on the workbench creation page.
    a. Click the **Software** tab.
    b. Click the **Add software** button.
    c. Click **Edit** ().
    d. Enter the **Software** name.
    e. Enter the software **Version**.
    f. Click **Confirm** () to confirm your entry.
    g. To add additional software, click **Add software**, complete the relevant fields, and confirm your entry.
9. Optional: Add packages to the workbench images. After the import has completed, the packages are added to the image's meta-data and displayed on the workbench creation page.
    a. Click the **Packages** tab.
    b. Click the **Add package** button.
    c. Click **Edit** ().
    d. Enter the **Package** name.
    e. Enter the package **Version**. For example, type **3.16.7**.
    f. Click **Confirm** () to confirm your entry.
    g. To add an additional package, click **Add package**, complete the relevant fields, and confirm your entry.
10. Click **Import**.

Verification

- The image that you imported is displayed in the table on the **Workbench images** page.
- Your custom image is available for selection when a user creates a workbench.

Additional resources

- Managing image streams
- Understanding build configurations

CHAPTER 3. MANAGING APPLICATIONS THAT SHOW IN THE DASHBOARD

3.1. ADDING AN APPLICATION TO THE DASHBOARD

If you have installed an application in your OpenShift cluster, an OpenShift AI administrator can add a tile for that application to the OpenShift AI dashboard (the **Applications → Enabled** page) to make it accessible for OpenShift AI users.

Prerequisites

- You have OpenShift AI administrator privileges.
- The **spec.dashboardConfig.enablement** dashboard configuration option is set to **true** (the default).
  For more information about setting dashboard configuration options, see Customizing the dashboard.

Procedure

1. Log in to the OpenShift console as an OpenShift AI administrator.
2. In the **Administrator** perspective, click **Home → API Explorer**.
3. In the search bar, enter **OdhApplication** to filter by kind.
4. Click the **OdhApplication** custom resource (CR) to open the resource details page.
5. From the **Project** list, select the OpenShift AI application namespace; the default is **redhat-ods-applications**.
6. Click the **Instances** tab.
7. Click **Create OdhApplication**.
8. On the **Create OdhApplication** page, copy the following code and paste it into the YAML editor.

```
apiVersion: dashboard.opendatahub.io/v1
kind: OdhApplication
metadata:
  name: examplename
  namespace: redhat-ods-applications
  labels:
    app: odh-dashboard
    app.kubernetes.io/part-of: odh-dashboard
spec:
  enable:
    validationConfigMap: examplename-enable
  img: >-
    <svg width="24" height="25" viewBox="0 0 24 25" fill="none"
xmlns="http://www.w3.org/2000/svg">
    <path d="path data" fill="#ee0000"/>
    </svg>
```

```
getStartedLink: 'https://example.org/docs/quickstart.html'
route: exampleroutename
routeNamespace: examplenamespace
displayName: Example Name
kfdefApplications: []
support: third party support
csvName: ''
provider: example
docsLink: 'https://example.org/docs/index.html'
quickStart: ''
getStartedMarkDown: >-
  # Example

  Enter text for the information panel.

description: >-
  Enter summary text for the tile.
category: Self-managed | Partner managed | Red Hat managed
```

9. Modify the parameters in the code for your application.

   TIP

   To see example YAML files, click **Home → API Explorer**, select **OdhApplication**, click the **Instances** tab, select an instance, and then click the **YAML** tab.

10. Click **Create**. The application details page opens.
11. Log in to OpenShift AI.
12. In the left menu, click **Applications → Explore**.
13. Locate the new tile for your application and click it.
14. In the information pane for the application, click **Enable**.

Verification

- In the left menu of the OpenShift AI dashboard, click **Applications → Enabled** and verify that your application is available.

3.2. PREVENTING USERS FROM ADDING APPLICATIONS TO THE DASHBOARD

By default, OpenShift AI administrators can add applications to the OpenShift AI dashboard **Application → Enabled** page.

As an OpenShift AI administrator, you can disable the ability for OpenShift AI administrators to add applications to the dashboard.

Note: The **Start basic workbench** tile is enabled by default. To disable it, see Hiding the default basic workbench application.

Prerequisite

- You have OpenShift AI administrator privileges.

Procedure

1. Log in to the OpenShift console as an OpenShift AI administrator.
2. Open the dashboard configuration file:
   a. In the **Administrator** perspective, click **Home → API Explorer**.
   b. In the search bar, enter **OdhDashboardConfig** to filter by kind.
   c. Click the **OdhDashboardConfig** custom resource (CR) to open the resource details page.
   d. From the **Project** list, select the OpenShift AI application namespace; the default is **redhat-ods-applications**.
   e. Click the **Instances** tab.
   f. Click the **odh-dashboard-config** instance to open the details page.
   g. Click the **YAML** tab.
3. In the **spec.dashboardConfig** section, set the value of **enablement** to **false** to disable the ability for dashboard users to add applications to the dashboard.
4. Click **Save** to apply your changes and then click **Reload** to make sure that your changes are synced to the cluster.

Verification

- Open the OpenShift AI dashboard **Application → Enabled** page.

3.3. DISABLING APPLICATIONS CONNECTED TO OPENSHIFT AI

You can disable applications and components so that they do not appear on the OpenShift AI dashboard when you no longer want to use them, for example, when data scientists no longer use an application or when the application license expires.

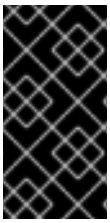
Disabling unused applications allows your data scientists to manually remove these application tiles from their OpenShift AI dashboard so that they can focus on the applications that they are most likely to use. See Removing disabled applications from the dashboard for more information about manually removing application tiles.

Prerequisites

- You have logged in to the OpenShift web console.
- You are part of the **cluster-admins** user group in OpenShift.
- You have installed or configured the service on your OpenShift cluster.
- The application or component that you want to disable is enabled and visible on the **Enabled** page.

Procedure

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Switch to the **redhat-ods-applications** project.
3. Click **Operators → Installed Operators**.
4. Click on the Operator that you want to uninstall. You can enter a keyword into the **Filter by name** field to help you find the Operator faster.
5. Delete any Operator resources or instances by using the tabs in the Operator interface. During installation, some Operators require the administrator to create resources or start process instances using tabs in the Operator interface. These must be deleted before the Operator can uninstall correctly.
6. On the **Operator Details** page, click the **Actions** drop-down menu and select **Uninstall Operator**.
   An **Uninstall Operator?** dialog box is displayed.
7. Select **Uninstall** to uninstall the Operator, Operator deployments, and pods. After this is complete, the Operator stops running and no longer receives updates.



IMPORTANT

Removing an Operator does not remove any custom resource definitions or managed resources for the Operator. Custom resource definitions and managed resources still exist and must be cleaned up manually. Any applications deployed by your Operator and any configured off-cluster resources continue to run and must be cleaned up manually.

Verification

- The Operator is uninstalled from its target clusters.
- The Operator is no longer displayed on the **Installed Operators** page.
- The disabled application is no longer available for your data scientists to use, and is marked as **Disabled** on the **Enabled** page of the OpenShift AI dashboard. This action may take a few minutes to occur following the removal of the Operator.

3.4. SHOWING OR HIDING INFORMATION ABOUT AVAILABLE APPLICATIONS

You can view a list of available applications in the **Exploring applications** page of the OpenShift AI dashboard. By default, the following information is provided for each application:

- Any independent software vendor (ISV) application is indicated with a label on the tile indicating **Red Hat-managed**, **Partner managed**, or **Self-managed**. As an OpenShift AI administrator, you can hide or show the labels. For example, if you are running a self-managed environment, you might want to show all available applications regardless of the support level.
- When a user clicks on an application, an information panel is displayed and provides more information about the application, including links to quick starts or detailed documentation. You can disable or enable the appearance of application information panels.

Prerequisites

- You have OpenShift AI administrator privileges.

Procedure

1. Log in to the OpenShift console as an OpenShift AI administrator.
2. Open the dashboard configuration file:
    a. In the **Administrator** perspective, click **Home → API Explorer**.
    b. In the search bar, enter **OdhDashboardConfig** to filter by kind.
    c. Click the **OdhDashboardConfig** custom resource (CR) to open the resource details page.
    d. From the **Project** list, select the OpenShift AI application namespace; the default is **redhat-ods-applications**.
    e. Click the **Instances** tab.
    f. Click the **odh-dashboard-config** instance to open the details page.
    g. Click the **YAML** tab.
3. In the **spec.dashboardConfig** section, set either or both of the following options:
    - **disableInfo**: Set to **true** to hide the appearance of application information panel. Set to **False** (the default) to show the application information panel.
    - **disableISVBadges**: Set to **true** to hide the appearance of the support-level label. Set to **False** (the default) to show the support-level label.
4. Click **Save** to apply your changes and then click **Reload** to make sure that your changes are synced to the cluster.

Verification

Log in to OpenShift AI and verify that your dashboard configurations apply.

3.5. HIDING THE DEFAULT BASIC WORKBENCH APPLICATION

The OpenShift AI dashboard includes **Start basic workbench** as an enabled application by default.

To hide the **Start basic workbench** tile so that it is no longer included in the list of applications on the **Applications → Enabled** page, edit the dashboard configuration file.

Prerequisite

- You have OpenShift AI administrator privileges.

Procedure

1. Log in to the OpenShift console as an OpenShift AI administrator.
2. Open the dashboard configuration file:

a. In the **Administrator** perspective, click **Home → API Explorer**.

b. In the search bar, enter **OdhDashboardConfig** to filter by kind.

c. Click the **OdhDashboardConfig** custom resource (CR) to open the resource details page.

d. From the **Project** list, select the OpenShift AI application namespace; the default is **redhat-ods-applications**.

e. Click the **Instances** tab.

f. Click the **odh-dashboard-config** instance to open the details page.

g. Click the **YAML** tab.

3. In the **spec:notebookController** section, set the value of **enabled** to **false** to remove the **Start basic workbench** tile from the list of applications on the **Applications → Enabled** page.

4. Click **Save** to apply your changes and then click **Reload** to make sure that your changes are synced to the cluster.

Verification

In the OpenShift AI dashboard, click **Applications → Enabled**. The list of applications no longer includes the **Start basic workbench** tile.

CHAPTER 4. CREATING PROJECT-SCOPED RESOURCES

OpenShift AI users can access *global resources* in all OpenShift AI projects. However, they can access *project-scoped resources* only within projects that they have permissions to access.

As a cluster administrator, you can create the following types of project-scoped resources in any OpenShift AI project:

- Workbench images
- Hardware profiles
- Model-serving runtimes for KServe

All resource names must be unique within a project.

NOTE

A user with access permissions to a project can create project-scoped resources for that project, as described in Creating project-scoped resources for your project .

Prerequisites

- You can access the OpenShift console as a cluster administrator.
- You have set the **disableProjectScoped** dashboard configuration option to **false**, as described in Customizing the dashboard.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. Copy the YAML code to create the resource.
   You can get the YAML code from a trusted source, such as an existing resource, a Git repository, or documentation.

   For example, you can copy the YAML code from an existing resource, as follows:

   a. In the **Administrator** perspective, click **Home → Search**.

   b. From the **Project** list, select the appropriate project.
   To limit the search to global OpenShift AI resources only, select the **redhat-ods-applications** project.

   c. In the **Resources** list, search for the relevant resource type:

   - For workbench images, search for **ImageStream**.
   - For hardware profiles, search for **HardwareProfile**.
   - For serving runtimes, search for **Template**. From the resulting list, find the templates that have the **objects.kind** specification set to **ServingRuntime**.

   d. Select a resource, and then click the **YAML** tab.

   e. Copy the YAML content, and then click **Cancel**.

3. From the **Project** list, select the target project name.
4. From the toolbar, click the **+** icon to open the **Import YAML** page.
5. Paste the relevant YAML content into the code area.
6. Edit the **metadata.namespace** value to specify the name of the target project.
7. If necessary, edit the **metadata.name** value to ensure that the resource name is unique within the specified project.
8. Optional: Edit the resource name that is displayed in the OpenShift AI console:
    - For workbench images, edit the **metadata.annotations.opendatahub.io/notebook-image-name** value.
    - For hardware profiles, edit the **spec.displayName** value.
    - For serving runtimes, edit the **objects.metadata.annotations.openshift.io/display-name** value.
9. Click **Create**.

Verification

1. Log in to the OpenShift AI console as a regular user.
2. Verify that the project-scoped resource is shown in the specified project:
    - For workbench images and hardware profiles, see Creating a workbench.
    - For serving runtimes, see Deploying models.

CHAPTER 5. ALLOCATING ADDITIONAL RESOURCES TO OPENSHIFT AI USERS

As a cluster administrator, you can allocate additional resources to a cluster to support compute-intensive data science work. This support includes increasing the number of nodes in the cluster and changing the cluster's allocated machine pool.

For more information about allocating additional resources to an OpenShift cluster, see Manually scaling a compute machine set.

CHAPTER 6. CUSTOMIZING COMPONENT DEPLOYMENT RESOURCES

6.1. OVERVIEW OF COMPONENT RESOURCE CUSTOMIZATION

You can customize deployment resources that are related to the Red Hat OpenShift AI Operator, for example, CPU and memory limits and requests. For resource customizations to persist without being overwritten by the Operator, the **opendatahub.io/managed: true** annotation must not be present in the YAML file for the component deployment. This annotation is absent by default.

The following table shows the deployment names for each component in the **redhat-ods-applications** namespace.

IMPORTANT

Components denoted with **(Technology Preview)** in this table are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using Technology Preview features in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process. For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

Component	Deployment names
KServe	• kserve-controller-manager • odh-model-controller
Ray	kuberay-operator
Kueue	kueue-controller-manager
Workbenches	• notebook-controller-deployment • odh-notebook-controller-manager
Dashboard	rhods-dashboard
Model serving	• modelmesh-controller • odh-model-controller
Model registry	model-registry-operator-controller-manager

Component	Deployment names
AI pipelines	data-science-pipelines-operator-controller-manager
Training Operator	kubeflow-training-operator

6.2. CUSTOMIZING COMPONENT RESOURCES

You can customize component deployment resources by updating the **.spec.template.spec.containers.resources** section of the YAML file for the component deployment.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. In the **Administrator** perspective, click **Workloads → Deployments**.
3. From the **Project** drop-down list, select **redhat-ods-applications**.
4. In the **Name** column, click the name of the deployment for the component that you want to customize resources for.

NOTE

For more information about the deployment names for each component, see Overview of component resource customization.

5. On the **Deployment details** page that is displayed, click the **YAML** tab.
6. Find the **.spec.template.spec.containers.resources** section.
7. Update the value of the resource that you want to customize. For example, to update the memory limit to 500Mi, make the following change:

```
containers:
      - resources:
          limits:
            cpu: '2'
            memory: 500Mi
          requests:
            cpu: '1'
            memory: 1Gi
```

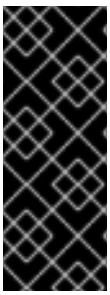
8. Click **Save**.
9. Click **Reload**.

Verification

- Log in to OpenShift AI and verify that your resource changes apply.

6.3. DISABLING COMPONENT RESOURCE CUSTOMIZATION

You can disable customization of component deployment resources, and restore default values, by adding the **opendatahub.io/managed: true** annotation to the YAML file for the component deployment.



IMPORTANT

Manually removing or setting the **opendatahub.io/managed: true** annotation to **false** after manually adding it to the YAML file for a component deployment might cause unexpected cluster issues.

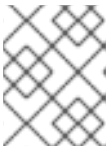
To remove the annotation from a deployment, use the steps described in Re-enabling component resource customization.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. In the **Administrator** perspective, click **Workloads → Deployments**.
3. From the **Project** drop-down list, select **redhat-ods-applications**.
4. In the **Name** column, click the name of the deployment for the component to which you want to add the annotation.



NOTE

For more information about the deployment names for each component, see Overview of component resource customization.

5. On the **Deployment details** page that opens, click the **YAML** tab.
6. Find the **metadata.annotations:** section.
7. Add the **opendatahub.io/managed: true** annotation.

```
metadata:
  annotations:
    opendatahub.io/managed: true
```

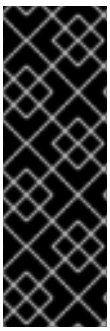
8. Click **Save**.
9. Click **Reload**.

Verification

- The **opendatahub.io/managed: true** annotation is displayed in the YAML file for the component deployment.

6.4. RE-ENABLING COMPONENT RESOURCE CUSTOMIZATION

You can re-enable customization of component deployment resources after manually disabling it.



IMPORTANT

Manually removing or setting the **opendatahub.io/managed:** annotation to **false** after adding it to the YAML file for a component deployment might cause unexpected cluster issues.

To remove the annotation from a deployment, use the following steps to delete the deployment. The controller pod for the deployment will automatically redeploy with the default settings.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. In the **Administrator** perspective, click **Workloads → Deployments**.
3. From the **Project** drop-down list, select **redhat-ods-applications**.
4. In the **Name** column, click the name of the deployment for the component for which you want to remove the annotation.
5. Click the Options menu .
6. Click **Delete Deployment**.

Verification

- The controller pod for the deployment automatically redeploys with the default settings.

CHAPTER 7. ENABLING ACCELERATORS

7.1. ENABLING NVIDIA GPUS

Before you can use NVIDIA GPUs in OpenShift AI, you must install the NVIDIA GPU Operator.

IMPORTANT

If you are using OpenShift AI in a disconnected self-managed environment, see Enabling accelerators instead.

Prerequisites

- You have logged in to your OpenShift cluster.
- You have the **cluster-admin** role in your OpenShift cluster.
- You have installed an NVIDIA GPU and confirmed that it is detected in your environment.

Procedure

1. To enable GPU support on an OpenShift cluster, follow the instructions here: NVIDIA GPU Operator on Red Hat OpenShift Container Platform in the NVIDIA documentation.

IMPORTANT

After you install the Node Feature Discovery (NFD) Operator, you must create an instance of NodeFeatureDiscovery. In addition, after you install the NVIDIA GPU Operator, you must create a ClusterPolicy and populate it with default values.

2. Delete the **migration-gpu-status** ConfigMap.
    a. In the OpenShift web console, switch to the **Administrator** perspective.
    b. Set the **Project** to **All Projects** or **redhat-ods-applications** to ensure you can see the appropriate ConfigMap.
    c. Search for the **migration-gpu-status** ConfigMap.
    d. Click the action menu (⋮) and select **Delete ConfigMap** from the list.
    The **Delete ConfigMap** dialog opens.
    e. Inspect the dialog and confirm that you are deleting the correct ConfigMap.
    f. Click **Delete**.
3. Restart the dashboard replicaset.
    a. In the OpenShift web console, switch to the **Administrator** perspective.
    b. Click **Workloads → Deployments**.
    c. Set the **Project** to **All Projects** or **redhat-ods-applications** to ensure you can see the appropriate deployment.

d. Search for the **rhods-dashboard** deployment.

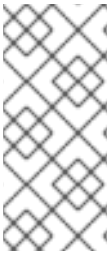
e. Click the action menu (⋮) and select **Restart Rollout** from the list.

f. Wait until the **Status** column indicates that all pods in the rollout have fully restarted.

Verification

- The reset **migration-gpu-status** instance is present on the **Instances** tab on the **AcceleratorProfile** custom resource definition (CRD) details page.
- From the **Administrator** perspective, go to the **Operators → Installed Operators** page. Confirm that the following Operators appear:
  - NVIDIA GPU
  - Node Feature Discovery (NFD)
  - Kernel Module Management (KMM)
- The GPU is correctly detected a few minutes after full installation of the Node Feature Discovery (NFD) and NVIDIA GPU Operators. The OpenShift CLI (**oc**) displays the appropriate output for the GPU worker node. For example:

```
# Expected output when the GPU is detected properly
oc describe node <node name>
...
Capacity:
  cpu:                4
  ephemeral-storage:  313981932Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             16076568Ki
  nvidia.com/gpu:     1
  pods:               250
Allocatable:
  cpu:                3920m
  ephemeral-storage:  288292006229
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             12828440Ki
  nvidia.com/gpu:     1
  pods:               250
```



NOTE

In OpenShift AI, Red Hat supports the use of accelerators within the same cluster only.

Starting from Red Hat OpenShift AI 2.19, Red Hat supports remote direct memory access (RDMA) for NVIDIA GPUs only, enabling them to communicate directly with each other by using NVIDIA GPUDirect RDMA across either Ethernet or InfiniBand networks.

After installing the NVIDIA GPU Operator, create a hardware profile as described in Working with hardware profiles.

7.2. INTEL GAUDI AI ACCELERATOR INTEGRATION

To accelerate your high-performance deep learning models, you can integrate Intel Gaudi AI accelerators into OpenShift AI. This integration enables your data scientists to use Gaudi libraries and software associated with Intel Gaudi AI accelerators through custom-configured workbench instances.

Intel Gaudi AI accelerators offer optimized performance for deep learning workloads, with the latest Gaudi 3 devices providing significant improvements in training speed and energy efficiency. These accelerators are suitable for enterprises running machine learning and AI applications on OpenShift AI.

Before you can enable Intel Gaudi AI accelerators in OpenShift AI, you must complete the following steps:

1. Install the latest version of the Intel Gaudi Base Operator from OperatorHub.
2. Create and configure a custom workbench image for Intel Gaudi AI accelerators. A prebuilt workbench image for Gaudi accelerators is not included in OpenShift AI.
3. Manually define and configure a hardware profile for each Intel Gaudi AI device in your environment.

Red Hat supports Intel Gaudi devices up to Intel Gaudi 3. The Intel Gaudi 3 accelerators, in particular, offer the following benefits:

- Improved training throughput: Reduce the time required to train large models by using advanced tensor processing cores and increased memory bandwidth.
- Energy efficiency: Lower power consumption while maintaining high performance, reducing operational costs for large-scale deployments.
- Scalable architecture: Scale across multiple nodes for distributed training configurations.

Your OpenShift platform must support EC2 DL1 instances to use Intel Gaudi AI accelerators in an Amazon EC2 DL1 instance. You can use Intel Gaudi AI accelerators in workbench instances or model serving after you enable the accelerators, create a custom workbench image, and configure the hardware profile.

To identify the Intel Gaudi AI accelerators present in your deployment, use the **lspci** utility. For more information, see lspci(8) - Linux man page .

IMPORTANT

The presence of Intel Gaudi AI accelerators in your deployment, as indicated by the **lspci** utility, does not guarantee that the devices are ready to use. You must ensure that all installation and configuration steps are completed successfully.

Additional resources

- lspci(8) - Linux man page
- Amazon EC2 DL1 Instances
- Intel Gaudi AI Operator OpenShift installation
- What version of the Kubernetes API is included with each OpenShift 4.x release?

7.2.1. Enabling Intel Gaudi AI accelerators

Before you can use Intel Gaudi AI accelerators in OpenShift AI, you must install the required dependencies, deploy the Intel Gaudi Base Operator, and configure the environment.

Prerequisites

- You have logged in to OpenShift.
- You have the **cluster-admin** role in OpenShift.
- You have installed your Intel Gaudi accelerator and confirmed that it is detected in your environment.
- Your OpenShift environment supports EC2 DL1 instances if you are running on Amazon Web Services (AWS).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. Install the latest version of the Intel Gaudi Base Operator, as described in Intel Gaudi Base Operator OpenShift installation.
2. By default, OpenShift sets a per-pod PID limit of 4096. If your workload requires more processing power, such as when you use multiple Gaudi accelerators or when using vLLM with Ray, you must manually increase the per-pod PID limit to avoid **Resource temporarily unavailable** errors. These errors occur due to PID exhaustion. Red Hat recommends setting this limit to 32768, although values over 20000 are sufficient.
   a. Run the following command to label the node:

   ```
   oc label node <node_name> custom-kubelet=set-pod-pid-limit-kubelet
   ```

   b. Optional: To prevent workload distribution on the affected node, you can mark the node as unschedulable and then drain it in preparation for maintenance. For more information, see Understanding how to evacuate pods on nodes .
   c. Create a **custom-kubelet-pidslimit.yaml** KubeletConfig resource file:

   ```
   oc create -f custom-kubelet-pidslimit.yaml
   ```

   d. Populate the file with the following YAML code. Set the **PodPidsLimit** value to 32768:

   ```
   apiVersion: machineconfiguration.openshift.io/v1
   kind: KubeletConfig
   metadata:
     name: custom-kubelet-pidslimit
   spec:
     kubeletConfig:
   ```

```
    PodPidsLimit: 32768
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: set-pod-pid-limit-kubelet
```

e. Apply the configuration:

```
oc apply -f custom-kubelet-pidslimit.yaml
```

This operation causes the node to reboot. For more information, see Understanding node rebooting.

f. Optional: If you previously marked the node as unschedulable, you can allow scheduling again after the node reboots.

3. Create a custom workbench image for Intel Gaudi AI accelerators, as described in Creating custom workbench images.

4. After installing the Intel Gaudi Base Operator, create a hardware profile, as described in Working with hardware profiles.

Verification

From the **Administrator** perspective, go to the **Operators → Installed Operators** page. Confirm that the following Operators appear:

- Intel Gaudi Base Operator
- Node Feature Discovery (NFD)
- Kernel Module Management (KMM)

7.3. AMD GPU INTEGRATION

You can use AMD GPUs with OpenShift AI to accelerate AI and machine learning (ML) workloads. AMD GPUs provide high-performance compute capabilities, allowing users to process large data sets, train deep neural networks, and perform complex inference tasks more efficiently.

Integrating AMD GPUs with OpenShift AI involves the following components:

- **ROCm workbench images**: Use the ROCm workbench images to streamline AI/ML workflows on AMD GPUs. These images include libraries and frameworks optimized with the AMD ROCm platform, enabling high-performance workloads for PyTorch and TensorFlow. The pre-configured images reduce setup time and provide an optimized environment for GPU-accelerated development and experimentation.
- **AMD GPU Operator**: The AMD GPU Operator simplifies GPU integration by automating driver installation, device plugin setup, and node labeling for GPU resource management. It ensures compatibility between OpenShift and AMD hardware while enabling scaling of GPU-enabled workloads.

7.3.1. Verifying AMD GPU availability on your cluster

Before you proceed with the AMD GPU Operator installation process, you can verify the presence of an AMD GPU device on a node within your OpenShift cluster. You can use commands such as **lspci** or **oc** to confirm hardware and resource availability.

Prerequisites

- You have administrative access to the OpenShift cluster.
- You have a running OpenShift cluster with a node equipped with an AMD GPU.
- You have access to the OpenShift CLI (**oc**) and terminal access to the node.

Procedure

1. Use the OpenShift CLI (**oc**) to verify if GPU resources are allocatable:
   a. List all nodes in the cluster to identify the node with an AMD GPU:

```
oc get nodes
```

   b. Note the name of the node where you expect the AMD GPU to be present.
   c. Describe the node to check its resource allocation:

```
oc describe node <node_name>
```

   d. In the output, locate the **Capacity** and **Allocatable** sections and confirm that **amd.com/gpu** is listed. For example:

```
Capacity:
  amd.com/gpu:  1
Allocatable:
  amd.com/gpu:  1
```

2. Check for the AMD GPU device using the **lspci** command:
   a. Log in to the node:

```
oc debug node/<node_name>
chroot /host
```

   b. Run the **lspci** command and search for the supported AMD device in your deployment. For example:

```
lspci | grep -E "MI210|MI250|MI300"
```

   c. Verify that the output includes one of the AMD GPU models. For example:

```
03:00.0 Display controller: Advanced Micro Devices, Inc. [AMD] Instinct MI210
```

3. Optional: Use the **rocminfo** command if the ROCm stack is installed on the node:

```
rocminfo
```

   a. Confirm that the ROCm tool outputs details about the AMD GPU, such as compute units, memory, and driver status.

Verification

- The **oc describe node <node_name>** command lists **amd.com/gpu** under **Capacity** and **Allocatable**.
- The **lspci** command output identifies an AMD GPU as a PCI device matching one of the specified models (for example, MI210, MI250, MI300).
- Optional: The **rocminfo** tool provides detailed GPU information, confirming driver and hardware configuration.

Additional resources

- AMD GPU Operator GitHub Repository

7.3.2. Enabling AMD GPUs

Before you can use AMD GPUs in OpenShift AI, you must install the required dependencies, deploy the AMD GPU Operator, and configure the environment.

Prerequisites

- You have logged in to OpenShift.
- You have the **cluster-admin** role in OpenShift.
- You have installed your AMD GPU and confirmed that it is detected in your environment.
- Your OpenShift environment supports EC2 DL1 instances if you are running on Amazon Web Services (AWS).

Procedure

1. Install the latest version of the AMD GPU Operator, as described in Install AMD GPU Operator on OpenShift.
2. After installing the AMD GPU Operator, configure the AMD drivers required by the Operator as described in the documentation: Configure AMD drivers for the GPU Operator .

NOTE

Alternatively, you can install the AMD GPU Operator from the Red Hat Catalog. For more information, see Install AMD GPU Operator from Red Hat Catalog .

1. After installing the AMD GPU Operator, create a hardware profile, as described in Working with hardware profiles.

Verification

From the **Administrator** perspective, go to the **Operators → Installed Operators** page. Confirm that the following Operators appear:

- AMD GPU Operator
- Node Feature Discovery (NFD)

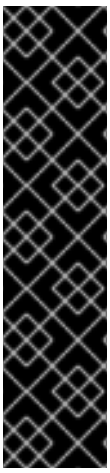
- Kernel Module Management (KMM)

NOTE

Ensure that you follow all the steps for proper driver installation and configuration. Incorrect installation or configuration may prevent the AMD GPUs from being recognized or functioning properly.

CHAPTER 8. MANAGING WORKLOADS WITH KUEUE

As a cluster administrator, you can manage AI and machine learning workloads at scale by integrating the Red Hat build of Kueue with Red Hat OpenShift AI. This integration provides capabilities for quota management, resource allocation, and prioritized job scheduling.



IMPORTANT

Starting with OpenShift AI 2.24, the embedded Kueue component for managing distributed workloads is deprecated. Kueue is now provided through Red Hat build of Kueue, which is installed and managed by the Red Hat build of Kueue Operator. You cannot install both the embedded Kueue and the Red Hat build of Kueue Operator on the same cluster because this creates conflicting controllers that manage the same resources.

OpenShift AI does not automatically migrate existing workloads. To ensure your workloads continue using queue management after upgrading, cluster administrators must manually migrate from the embedded Kueue to the Red Hat build of Kueue Operator. For more information, see Migrating to the Red Hat build of Kueue Operator .

8.1. OVERVIEW OF MANAGING WORKLOADS WITH KUEUE

You can use Kueue in OpenShift AI to manage AI and machine learning workloads at scale. Kueue controls how cluster resources are allocated and shared through hierarchical quota management, dynamic resource allocation, and prioritized job scheduling. These capabilities help prevent cluster contention, ensure fair access across teams, and optimize the use of heterogeneous compute resources, such as hardware accelerators.

Kueue lets you schedule diverse workloads, including distributed training jobs (**RayJob**, **RayCluster**, **PyTorchJob**), workbenches (**Notebook**), and model serving (**InferenceService**). Kueue validation and queue enforcement apply only to workloads in namespaces with the **kueue.openshift.io/managed=true** label.

Using Kueue in OpenShift AI provides these benefits:

- Prevents resource conflicts and prioritizes workload processing
- Manages quotas across teams and projects
- Ensures consistent scheduling for all workload types
- Maximizes GPU and other specialized hardware utilization

IMPORTANT

Starting with OpenShift AI 2.24, the embedded Kueue component for managing distributed workloads is deprecated. Kueue is now provided through Red Hat build of Kueue, which is installed and managed by the Red Hat build of Kueue Operator. You cannot install both the embedded Kueue and the Red Hat build of Kueue Operator on the same cluster because this creates conflicting controllers that manage the same resources.

OpenShift AI does not automatically migrate existing workloads. To ensure your workloads continue using queue management after upgrading, cluster administrators must manually migrate from the embedded Kueue to the Red Hat build of Kueue Operator. For more information, see Migrating to the Red Hat build of Kueue Operator .

8.1.1. Kueue management states

You configure how OpenShift AI interacts with Kueue by setting the **managementState** in the **DataScienceCluster** object.

Unmanaged

This state is supported for using Kueue with OpenShift AI. In **Unmanaged** state, OpenShift AI integrates with an existing Kueue installation managed by the Red Hat build of Kueue Operator. You must have the Red Hat build of Kueue Operator installed and running on the cluster.
When you enable **Unmanaged** mode, the OpenShift AI Operator creates a default **Kueue** custom resource (CR) if one does not already exist. This prompts the Red Hat build of Kueue Operator to activate Kueue on the cluster.

Managed

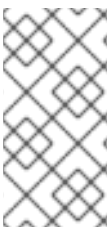
This state is deprecated. Previously, OpenShift AI deployed and managed an embedded Kueue distribution. **Managed** mode is not compatible with the Red Hat build of Kueue Operator. If both are installed, OpenShift AI stops reconciliation to avoid conflicts. You must migrate any environment using the **Managed** state to the **Unmanaged** state to ensure continued support.

Removed

This state disables Kueue in OpenShift AI. If the state was previously **Managed**, OpenShift AI uninstalls the embedded Kueue distribution. If the state was previously **Unmanaged**, OpenShift AI stops checking for the external Kueue integration but does not uninstall the Red Hat build of Kueue Operator. An empty **managementState** value also functions as **Removed**.

8.1.2. Queue enforcement for projects

To ensure workloads do not bypass the queuing system, a validating webhook automatically enforces queuing rules on any project that is enabled for Kueue management. You enable a project for Kueue management by applying the **kueue.openshift.io/managed=true** label to the project namespace.



NOTE

This validating webhook enforcement method replaces the Validating Admission Policy that was used with the deprecated embedded Kueue component. The system also supports the legacy **kueue-managed** label for backward compatibility, but **kueue.openshift.io/managed=true** is the recommended label going forward.

After a project is enabled for Kueue management, the webhook requires that any new or updated workload has the **kueue.x-k8s.io/queue-name** label. If this label is missing, the webhook prevents the workload from being created or updated.

OpenShift AI creates a default, cluster-scoped **ClusterQueue** (if one does not already exist) and a namespace-scoped **LocalQueue** for that namespace (if one does not already exist). These default resources are created with the **opendatahub.io/managed=false** annotation, so they are not managed after creation. Cluster administrators can change or delete them.

The webhook enforces this rule on the **create** and **update** operations for the following resource types:

- **InferenceService**
- **Notebook**
- **PyTorchJob**
- **RayCluster**
- **RayJob**

NOTE

You can apply hardware profiles to other workload types, but the validation webhook enforces the **kueue.x-k8s.io/queue-name** label requirement only for these specific resource types.

8.1.3. Restrictions for managing workloads with Kueue

When you use Kueue to manage workloads in OpenShift AI, the following restrictions apply:

- Namespaces must be labeled with **kueue.openshift.io/managed=true** to enable Kueue validation and queue enforcement.
- All workloads that you create from the OpenShift AI dashboard, such as workbenches and model servers, must use a hardware profile that specifies a local queue.
- When you specify a local queue in a hardware profile, OpenShift AI automatically applies the corresponding **kueue.x-k8s.io/queue-name** label to workloads that use that profile.
- You cannot use hardware profiles that contain node selectors or tolerations for node placement. To direct workloads to specific nodes, use a hardware profile that specifies a local queue that is associated with a queue configured with the appropriate resource flavors.
- Because workbenches are not suspendable workloads, you can only assign them to a local queue that is associated with a non-preemptive cluster queue. The default cluster queue that OpenShift AI creates is non-preemptive.

Additional resources

- Red Hat build of Kueue documentation

8.1.4. Kueue workflow

Managing workloads with Kueue in OpenShift AI involves tasks for OpenShift cluster administrators, OpenShift AI administrators, and machine learning (ML) engineers or data scientists:

Cluster administrator

Installs and configures Kueue:

1. Installs the Red Hat build of Kueue Operator on the cluster, as described in the Red Hat build of Kueue documentation.
2. Activates the Kueue integration by setting the **managementState** to **Unmanaged** in the **DataScienceCluster** custom resource, as described in Configuring workload management with Kueue.
3. Configures quotas to optimize resource allocation for user workloads, as described in the Red Hat build of Kueue documentation.
4. Enables Kueue in the dashboard by setting **disableKueue** to **false** in the **OdhDashboardConfig** custom resource, as described in Enabling Kueue in the dashboard.

NOTE

When Kueue is enabled in the dashboard, OpenShift AI automatically enables Kueue management for all new projects created from the dashboard. For existing projects, or for projects created by using the OpenShift CLI (**oc**), you must enable Kueue management manually by applying the **kueue.openshift.io/managed=true** label to the project namespace.

OpenShift AI administrator

Prepares the OpenShift AI environment:

1. Creates Kueue-enabled hardware profiles so that users can submit workloads from the OpenShift AI dashboard, as described in Working with hardware profiles.

ML Engineer or data scientist

Submits workloads to the queuing system:

1. For workloads created from the OpenShift AI dashboard, such as workbenches and model servers, selects a Kueue-enabled hardware profile during creation.
2. For workloads created by using a command-line interface or an SDK, such as distributed training jobs, adds the **kueue.x-k8s.io/queue-name** label to the workload's YAML manifest and sets its value to the target **LocalQueue** name.

8.2. CONFIGURING WORKLOAD MANAGEMENT WITH KUEUE

To use workload queuing in OpenShift AI, install the Red Hat build of Kueue Operator and activate the Kueue integration in OpenShift AI.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

- You are using OpenShift 4.19 or later.
- You have installed and configured the **cert-manager Operator for Red Hat OpenShift** for your cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Install the Red Hat build of Kueue Operator on your OpenShift cluster as described in the Red Hat build of Kueue documentation.
3. Activate the Kueue integration. You can use the predefined names for the default cluster queue and default local queue, or specify custom names.
   - To use the predefined queue names (**default**), run the following command. Replace **<operator-namespace>** with your operator namespace. The default operator namespace is **redhat-ods-operator**.

```
$ oc patch datasciencecluster default-dsc --type='merge' -p '{"spec":{"components":
{"kueue":{"managementState":"Unmanaged"}}}}' -n <operator-namespace>
```

   - To specify custom queue names, run the following command. Replace **<example-cluster-queue>** and **<example-local-queue>** with your custom queue names, and replace **<operator-namespace>** with your operator namespace. The default operator namespace is **redhat-ods-operator**.

```
$ oc patch datasciencecluster default-dsc --type='merge' -p '{"spec":{"components":
{"kueue":{"managementState":"Unmanaged","defaultClusterQueueName":"<example-
cluster-queue>","defaultLocalQueueName":"<example-local-queue>"}}}}' -n <operator-
namespace>
```

Verification

1. Verify that the Red Hat build of Kueue pods are running:

```
$ oc get pods -n openshift-kueue-operator
```

   You should see output similar to the following example:

```
kueue-controller-manager-d9fc745df-ph77w    1/1     Running
openshift-kueue-operator-69cfbf45cf-lwtpm   1/1     Running
```

2. Verify that the default **ClusterQueue** was created:

```
$ oc get clusterqueues
```

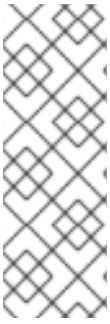
Next steps

- Configure quotas by creating and modifying **ResourceFlavor**, **ClusterQueue**, and **LocalQueue** objects. For details, see the Red Hat build of Kueue documentation.
- Enable Kueue in the dashboard so that users can select Kueue-enabled options when creating workloads. When you enable Kueue, you also enable Kueue management for all new projects created from the dashboard. See Enabling Kueue in the dashboard.
- Cluster administrators and OpenShift AI administrators can create hardware profiles so that users can submit workloads from the OpenShift AI dashboard. See Working with hardware profiles.

8.2.1. Enabling Kueue in the dashboard

Enable Kueue in the OpenShift AI dashboard so that users can select Kueue-enabled options when creating workloads.

When you enable Kueue in the dashboard, OpenShift AI automatically enables Kueue management for all new projects created from the dashboard. For these projects, OpenShift AI applies the **kueue.openshift.io/managed=true** label to the namespace and creates a **LocalQueue** object if one does not already exist. The **LocalQueue** object is created with the **opendatahub.io/managed=false** annotation, so it is not managed after creation. Cluster administrators can modify or delete it as needed. A validating webhook then enforces that any new or updated workload resource in a Kueue-enabled project includes the **kueue.x-k8s.io/queue-name** label.



NOTE

For existing projects, or for projects created by using the OpenShift CLI (**oc**), you must enable Kueue management manually by applying the **kueue.openshift.io/managed=true** label to the project namespace.

```
$ oc label namespace <project-namespace> kueue.openshift.io/managed=true --
overwrite
```

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You are using OpenShift 4.19 or later.
- You have installed and activated the Red Hat build of Kueue Operator, as described in Configuring workload management with Kueue.
- You have configured quotas, as described in the Red Hat build of Kueue documentation.

Procedure

1. In a terminal window, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Update the **odh-dashboard-config** custom resource in the OpenShift AI applications namespace. Replace **<applications-namespace>** with your OpenShift AI applications namespace. The default is **redhat-ods-applications**.

```
$ oc patch odhdashboardconfig odh-dashboard-config \
  -n \<applications-namespace\> \
  --type merge \
  -p {"spec":{"dashboardConfig":{"disableHardwareProfiles":false,"disableKueue":false}}}
```

Verification

1. From the OpenShift AI dashboard, create a new project.
2. Verify that the project namespace is labeled for Kueue management:

```
$ oc get ns <project-namespace> -o
jsonpath='{.metadata.labels.kueue\.openshift\.io/managed}{"\n"}'
```

   The output should be **true**.

3. Confirm that a default **LocalQueue** exists for the project namespace:

```
$ oc get localqueues -n <project-namespace>
```

4. Create a test workload (for example, a **Notebook**) and verify that it includes the **kueue.x-k8s.io/queue-name** label.

Next step

- Cluster administrators and OpenShift AI administrators can create hardware profiles so that users can submit workloads from the OpenShift AI dashboard. See Working with hardware profiles.

8.3. TROUBLESHOOTING COMMON PROBLEMS WITH KUEUE

If your users are experiencing errors in Red Hat OpenShift AI relating to Kueue workloads, read this section to understand what could be causing the problem, and how to resolve the problem.

If the problem is not documented here or in the release notes, contact Red Hat Support.

8.3.1. A user receives a "failed to call webhook" error message for Kueue

Problem

After the user runs the **cluster.apply()** command, the following error is shown:

```
ApiException: (500)
Reason: Internal Server Error
HTTP response body: {"kind":"Status","apiVersion":"v1","metadata":
{},"status":"Failure","message":"Internal error occurred: failed calling webhook \"mraycluster.kb.io\":
failed to call webhook: Post \"https://kueue-webhook-service.redhat-ods-applications.svc:443/mutate-
ray-io-v1-raycluster?timeout=10s\": no endpoints available for service \"kueue-webhook-
service\"","reason":"InternalError","details":{"causes":[{"message":"failed calling webhook
```

\"mraycluster.kb.io\": failed to call webhook: Post \"https://kueue-webhook-service.redhat-ods-applications.svc:443/mutate-ray-io-v1-raycluster?timeout=10s\": no endpoints available for service \"kueue-webhook-service\""}]},"code":500}

Diagnosis

The Kueue pod might not be running.

Resolution

1. In the OpenShift console, select the user's project from the **Project** list.
2. Click **Workloads → Pods**.
3. Verify that the Kueue pod is running. If necessary, restart the Kueue pod.
4. Review the logs for the Kueue pod to verify that the webhook server is serving, as shown in the following example:

{"level":"info","ts":"2024-06-24T14:36:24.255137871Z","logger":"controller-runtime.webhook","caller":"webhook/server.go:242","msg":"Serving webhook server","host":"","port":9443}

8.3.2. A user receives a "Default Local Queue ... not found" error message

Problem

After the user runs the **cluster.apply()** command, the following error is shown:

> Default Local Queue with kueue.x-k8s.io/default-queue: true annotation not found please create a default Local Queue or provide the local_queue name in Cluster Configuration.

Diagnosis

No default local queue is defined, and a local queue is not specified in the cluster configuration.

Resolution

1. Check whether a local queue exists in the user's project, as follows:
   a. In the OpenShift console, select the user's project from the **Project** list.
   b. Click **Home → Search**, and from the **Resources** list, select **LocalQueue**.
   c. If no local queues are found, create a local queue.
   d. Provide the user with the details of the local queues in their project, and advise them to add a local queue to their cluster configuration.
2. Define a default local queue.
   For information about creating a local queue and defining a default local queue, see Configuring quota management for distributed workloads.

8.3.3. A user receives a "local_queue provided does not exist" error message

Problem

After the user runs the **cluster.apply()** command, the following error is shown:

> local_queue provided does not exist or is not in this namespace. Please provide the correct local_queue name in Cluster Configuration.

Diagnosis

An incorrect value is specified for the local queue in the cluster configuration, or an incorrect default local queue is defined. The specified local queue either does not exist, or exists in a different namespace.

Resolution

a. In the OpenShift console, select the user's project from the **Project** list.

   1. Click **Search**, and from the **Resources** list, select **LocalQueue**.

   2. Resolve the problem in one of the following ways:

      - If no local queues are found, create a local queue.

      - If one or more local queues are found, provide the user with the details of the local queues in their project. Advise the user to ensure that they spelled the local queue name correctly in their cluster configuration, and that the **namespace** value in the cluster configuration matches their project name.

   3. Define a default local queue.
      For information about creating a local queue and defining a default local queue, see Configuring quota management for distributed workloads.

8.3.4. The pod provisioned by Kueue is terminated before the image is pulled

Problem

Kueue waits for a period of time before marking a workload as ready for all of the workload pods to become provisioned and running. By default, Kueue waits for 5 minutes. If the pod image is very large and is still being pulled after the 5-minute waiting period elapses, Kueue fails the workload and terminates the related pods.

Diagnosis

1. In the OpenShift console, select the user's project from the **Project** list.

2. Click **Workloads → Pods**.

3. Click the user's pod name to open the pod details page.

4. Click the **Events** tab, and review the pod events to check whether the image pull completed successfully.

Resolution

If the pod takes more than 5 minutes to pull the image, resolve the problem in one of the following ways:

- Add an **OnFailure** restart policy for resources that are managed by Kueue.

- Configure a custom timeout for the **waitForPodsReady** property in the **Kueue** custom resource (CR). The CR is installed in the **openshift-kueue-operator** namespace by the Red Hat build of Kueue Operator.

For more information about this configuration option, see Enabling waitForPodsReady in the Kueue documentation.

8.3.5. Additional resources

- Troubleshooting common problems with distributed workloads for administrators
- Troubleshooting common problems with distributed workloads for users

8.4. MIGRATING TO THE RED HAT BUILD OF KUEUE OPERATOR

Starting with OpenShift AI 2.24, the embedded Kueue component for managing distributed workloads is deprecated.

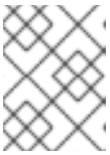
OpenShift AI now uses the Red Hat build of Kueue Operator to provide enhanced workload scheduling for distributed training, workbench, and model serving workloads.

Check if your environment is using the embedded Kueue component by verifying the **spec.components.kueue.managementState** field in the **DataScienceCluster** custom resource. If the field is set to **Managed**, you must migrate to the Red Hat build of Kueue Operator before upgrading OpenShift AI to avoid controller conflicts and ensure continued support for queue-based workloads.

OpenShift AI does not automatically migrate workloads, and you cannot install both the embedded Kueue and the Red Hat build of Kueue Operator on the same cluster.

Prerequisites

- Your environment is currently using the embedded Kueue component. That is, the **spec.components.kueue.managementState** field in the **DataScienceCluster** custom resource is set to **Managed**.



NOTE

If **spec.components.kueue.managementState** is set to **Removed** or **Unmanaged**, skip this migration.

- You have cluster administrator privileges for your OpenShift cluster.
- You are using OpenShift 4.19 or later.
- You have installed and configured the **cert-manager Operator for Red Hat OpenShift** for your cluster.

Procedure

1. Optional: When you migrate from the embedded Kueue to Red Hat build of Kueue, the OpenShift AI Operator automatically moves your existing Kueue configuration from the **kueue-manager-config** ConfigMap to the **Kueue** custom resource (CR).

If you want to keep the **kueue-manager-config** ConfigMap for reference, run the following command. Replace **<applications-namespace>** with your OpenShift AI applications namespace. The default namespace is **redhat-ods-applications**.

```
$ oc annotate configmap kueue-manager-config -n <applications-namespace>
opendatahub.io/managed=false
```

2. Log in to the OpenShift web console as a cluster administrator.
3. Uninstall the embedded Kueue component to avoid potential configuration conflicts.

NOTE

If you need to keep workloads running without interruption, you can skip this step. However, skipping it is not recommended because it might cause temporary configuration issues during the OpenShift AI upgrade.

   a. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
   b. Click the **Data Science Cluster** tab.
   c. Click the **default-dsc** object.
   d. Click the **YAML** tab.
   e. Set **spec.components.kueue.managementState** to **Removed** as shown:

```
spec:
  components:
    kueue:
      managementState: Removed
```

   f. Click **Save**.
   g. Wait for the OpenShift AI Operator to reconcile, and then verify that the embedded Kueue was removed:
      - On the **Details** tab of the **default-dsc** object, check that the **KueueReady** condition has a **Status** of **False** and a **Reason** of **Removed**.
      - Go to **Workloads → Deployments**, select the project where OpenShift AI is installed (for example, **redhat-ods-applications**), and confirm that Kueue-related deployments (for example, **kueue-controller-manager**) are no longer present.
4. Install the Red Hat build of Kueue Operator on your OpenShift cluster:
   a. Follow the steps to install the Red Hat build of Kueue Operator, as described in the Red Hat build of Kueue documentation.
   b. Go to **Operators → Installed Operators** and confirm that the Red Hat build of Kueue Operator is listed with **Status** as **Succeeded**.
5. Activate the Red Hat build of Kueue Operator in OpenShift AI:

a. In the web console, click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.

b. Click the **Data Science Cluster** tab.

c. Click the **default-dsc** object.

d. Click the **YAML** tab.

e. Set **spec.components.kueue.managementState** to **Unmanaged**. You can either use the predefined names (**default**) for the default cluster queue and default local queue, or specify custom names, as shown in the following examples.

- To use the predefined queue names, apply the following configuration:

```
spec:
  components:
    kueue:
      managementState: Unmanaged
```

- To specify custom queue names, apply the following configuration, replacing **<example-cluster-queue>** and **<example-local-queue>** with your custom values:

```
spec:
  components:
    kueue:
      managementState: Unmanaged
      defaultClusterQueueName: <example-cluster-queue>
      defaultLocalQueueName: <example-local-queue>
```

f. Click **Save**.

6. Enable Kueue management for existing projects by applying the **kueue.openshift.io/managed=true** label to each project namespace:

```
$ oc label namespace <project-namespace> kueue.openshift.io/managed=true --overwrite
```

Replace **<project-namespace>** with the name of your project.

NOTE

Kueue validation and queue enforcement apply only to workloads in namespaces labeled with **kueue.openshift.io/managed=true**.

Verification

- Verify that the embedded Kueue component was removed.
- Verify that the **DataScienceCluster** resource shows a healthy **Unmanaged** status for Kueue.
- Verify that existing workloads in the queue continue to be processed by the Red Hat build of Kueue controllers. Submit a new test workload to confirm functionality.

Next steps

- Configure quotas by creating and modifying **ResourceFlavor**, **ClusterQueue**, and **LocalQueue** objects. For details, see the Red Hat build of Kueue documentation.
- Enable Kueue in the dashboard so that users can select Kueue-enabled options when creating workloads. When enabled, Kueue management is automatically applied to all new projects created from the dashboard. See Enabling Kueue in the dashboard.
- Cluster administrators and OpenShift AI administrators can create hardware profiles so that users can submit workloads from the OpenShift AI dashboard. See Working with hardware profiles.

CHAPTER 9. MANAGING DISTRIBUTED WORKLOADS

In OpenShift AI, distributed workloads like **PyTorchJob**, **RayJob**, and **RayCluster** are created and managed by their respective workload operators. Kueue provides queueing and admission control and integrates with these operators to decide when workloads can run based on cluster-wide quotas.

You can perform advanced configuration for your distributed workloads environment, such as configuring quota management or setting up a cluster for RDMA.

9.1. CONFIGURING QUOTA MANAGEMENT FOR DISTRIBUTED WORKLOADS

Configure quotas for distributed workloads by creating Kueue resources. Quotas ensure that you can share resources between several projects.

Prerequisites

- You have logged in to OpenShift with the **cluster-admin** role.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have installed and activated the Red Hat build of Kueue Operator as described in Configuring workload management with Kueue.
- You have installed the required distributed workloads components as described in Installing the distributed workloads components (for disconnected environments, see Installing the distributed workloads components).
- You have created a project that contains a workbench, and the workbench is running a default workbench image that contains the CodeFlare SDK, for example, the **Standard Data Science** workbench. For information about how to create a project, see Creating a project.
- You have sufficient resources. In addition to the base OpenShift AI resources, you need 1.6 vCPU and 2 GiB memory to deploy the distributed workloads infrastructure.
- The resources are physically available in the cluster. For more information about Kueue resources, see the Red Hat build of Kueue documentation.
- If you want to use graphics processing units (GPUs), you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see Enabling NVIDIA GPUs. If you use AMD GPUs, see AMD GPU integration.

NOTE

In OpenShift AI 3.0, Red Hat supports only NVIDIA GPU accelerators and AMD GPU accelerators for distributed workloads.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Verify that a resource flavor exists or create a custom one, as follows:

   a. Check whether a **ResourceFlavor** already exists:

```
$ oc get resourceflavors
```

   b. If a **ResourceFlavor** already exists and you need to modify it, edit it in place:

```
$ oc edit resourceflavor <existing_resourceflavor_name>
```

   c. If a **ResourceFlavor** does not exist or you want a custom one, create a file called **default_flavor.yaml** and populate it with the following content:

   Empty Kueue resource flavor

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: <example_resource_flavor>
```

   For more examples, see *Example Kueue resource configurations*.

   d. Perform one of the following actions:

   - If you are modifying the existing resource flavor, save the changes.
   - If you are creating a new resource flavor, apply the configuration to create the **ResourceFlavor** object:

```
$ oc apply -f default_flavor.yaml
```

3. Verify that a default cluster queue exists or create a custom one, as follows:

NOTE

OpenShift AI automatically created a default cluster queue when the Kueue integration was activated. You can verify and modify the default cluster queue, or create a custom one.

   a. Check whether a **ClusterQueue** already exists:

```
$ oc get clusterqueues
```

   b. If a **ClusterQueue** already exists and you need to modify it (for example, to change the resources), edit it in place:

```
$ oc edit clusterqueue <existing_clusterqueue_name>
```

c. If a **ClusterQueue** does not exist or you want a custom one, create a file called **cluster_queue.yaml** and populate it with the following content:

Example cluster queue

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: <example_cluster_queue>
spec:
  namespaceSelector: {} (1)
  resourceGroups:
  - coveredResources: ["cpu", "memory", "nvidia.com/gpu"] (2)
    flavors:
    - name: "<resource_flavor_name>" (3)
      resources: (4)
      - name: "cpu"
        nominalQuota: 9
      - name: "memory"
        nominalQuota: 36Gi
      - name: "nvidia.com/gpu"
        nominalQuota: 5
```

1. Defines which namespaces can use the resources governed by this cluster queue. An empty **namespaceSelector** as shown in the example means that all namespaces can use these resources.
2. Defines the resource types governed by the cluster queue. This example **ClusterQueue** object governs CPU, memory, and GPU resources. If you use AMD GPUs, replace **nvidia.com/gpu** with **amd.com/gpu** in the example code.
3. Defines the resource flavor that is applied to the resource types listed. In this example, the <resource_flavor_name> resource flavor is applied to CPU, memory, and GPU resources.
4. Defines the resource requirements for admitting jobs. The cluster queue will start a distributed workload only if the total required resources are within these quota limits.

d. Replace the example quota values (9 CPUs, 36 GiB memory, and 5 NVIDIA GPUs) with the appropriate values for your cluster queue. If you use AMD GPUs, replace **nvidia.com/gpu** with **amd.com/gpu** in the example code. For more examples, see *Example Kueue resource configurations*.
You must specify a quota for each resource that the user can request, even if the requested value is 0, by updating the **spec.resourceGroups** section as follows:

- Include the resource name in the **coveredResources** list.
- Specify the resource **name** and **nominalQuota** in the **flavors.resources** section, even if the **nominalQuota** value is 0.

e. Perform one of the following actions:

- If you are modifying the existing cluster queue, save the changes.

- If you are creating a new cluster queue, apply the configuration to create the **ClusterQueue** object:

```
$ oc apply -f cluster_queue.yaml
```

4. Verify that a local queue that points to your cluster queue exists for your project namespace, or create a custom one, as follows:

NOTE

If Kueue is enabled in the OpenShift AI dashboard, new projects created from the dashboard are automatically configured for Kueue management. In those namespaces, a default local queue might already exist. You can verify and modify the local queue, or create a custom one.

a. Check whether a **LocalQueue** already exists for your project namespace:

```
$ oc get localqueues -n <project_namespace>
```

b. If a **LocalQueue** already exists and you need to modify it (for example, to point to a different **ClusterQueue**), edit it in place:

```
$ oc edit localqueue <existing_localqueue_name> -n <project_namespace>
```

c. If a **LocalQueue** does not exist or you want a custom one, create a file called **local_queue.yaml** and populate it with the following content:

Example local queue

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  name: <example_local_queue>
  namespace: <project_namespace>
spec:
  clusterQueue: <cluster_queue_name>
```

d. Replace the **name**, **namespace**, and **clusterQueue** values accordingly.

e. Perform one of the following actions:

- If you are modifying an existing local queue, save the changes.
- If you are creating a new local queue, apply the configuration to create the **LocalQueue** object:

```
$ oc apply -f local_queue.yaml
```

Verification

Check the status of the local queue in a project, as follows:

```
$ oc get localqueues -n <project_namespace>
```

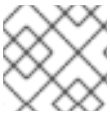
Additional resources

- Red Hat build of Kueue documentation
- Kueue documentation

9.2. EXAMPLE KUEUE RESOURCE CONFIGURATIONS FOR DISTRIBUTED WORKLOADS

You can use these example configurations as a starting point for creating Kueue resources to manage your distributed training workloads.

These examples show how to configure Kueue resource flavors and cluster queues for common distributed training scenarios.



NOTE

In OpenShift AI 3.0, Red Hat does not support shared cohorts.

9.2.1. NVIDIA GPUs without shared cohort

9.2.1.1. NVIDIA RTX A400 GPU resource flavor

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: "a400node"
spec:
  nodeLabels:
    instance-type: nvidia-a400-node
  tolerations:
  - key: "HasGPU"
    operator: "Exists"
    effect: "NoSchedule"
```

9.2.1.2. NVIDIA RTX A1000 GPU resource flavor

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: "a1000node"
spec:
  nodeLabels:
    instance-type: nvidia-a1000-node
  tolerations:
  - key: "HasGPU"
    operator: "Exists"
    effect: "NoSchedule"
```

9.2.1.3. NVIDIA RTX A400 GPU cluster queue

```
apiVersion: kueue.x-k8s.io/v1beta1
```

```
kind: ClusterQueue
metadata:
  name: "a400queue"
spec:
  namespaceSelector: {} # match all.
  resourceGroups:
  - coveredResources: ["cpu", "memory", "nvidia.com/gpu"]
    flavors:
    - name: "a400node"
      resources:
      - name: "cpu"
        nominalQuota: 16
      - name: "memory"
        nominalQuota: 64Gi
      - name: "nvidia.com/gpu"
        nominalQuota: 2
```

9.2.1.4. NVIDIA RTX A1000 GPU cluster queue

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: "a1000queue"
spec:
  namespaceSelector: {} # match all.
  resourceGroups:
  - coveredResources: ["cpu", "memory", "nvidia.com/gpu"]
    flavors:
    - name: "a1000node"
      resources:
      - name: "cpu"
        nominalQuota: 16
      - name: "memory"
        nominalQuota: 64Gi
      - name: "nvidia.com/gpu"
        nominalQuota: 2
```

9.2.2. NVIDIA GPUs and AMD GPUs without shared cohort

9.2.2.1. AMD GPU resource flavor

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: "amd-node"
spec:
  nodeLabels:
    instance-type: amd-node
  tolerations:
  - key: "HasGPU"
    operator: "Exists"
    effect: "NoSchedule"
```

9.2.2.2. NVIDIA GPU resource flavor

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: "nvidia-node"
spec:
  nodeLabels:
    instance-type: nvidia-node
  tolerations:
  - key: "HasGPU"
    operator: "Exists"
    effect: "NoSchedule"
```

9.2.2.3. AMD GPU cluster queue

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: "team-a-amd-queue"
spec:
  namespaceSelector: {} # match all.
  resourceGroups:
  - coveredResources: ["cpu", "memory", "amd.com/gpu"]
    flavors:
    - name: "amd-node"
      resources:
      - name: "cpu"
        nominalQuota: 16
      - name: "memory"
        nominalQuota: 64Gi
      - name: "amd.com/gpu"
        nominalQuota: 2
```

9.2.2.4. NVIDIA GPU cluster queue

```
apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: "team-a-nvidia-queue"
spec:
  namespaceSelector: {} # match all.
  resourceGroups:
  - coveredResources: ["cpu", "memory", "nvidia.com/gpu"]
    flavors:
    - name: "nvidia-node"
      resources:
      - name: "cpu"
        nominalQuota: 16
      - name: "memory"
        nominalQuota: 64Gi
      - name: "nvidia.com/gpu"
        nominalQuota: 2
```

Additional resources

- Red Hat build of Kueue documentation
- Resource Flavor in the Kueue documentation
- Cluster Queue in the Kueue documentation

9.3. CONFIGURING A CLUSTER FOR RDMA

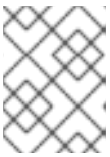
NVIDIA GPUDirect RDMA uses Remote Direct Memory Access (RDMA) to provide direct GPU interconnect. To configure a cluster for RDMA, a cluster administrator must install and configure several Operators.

Prerequisites

- You can access an OpenShift cluster as a cluster administrator.
- Your cluster has multiple worker nodes with supported NVIDIA GPUs, and can access a compatible NVIDIA accelerated networking platform.
- You have installed Red Hat OpenShift AI with the required distributed training components as described in Installing the distributed workloads components (for disconnected environments, see Installing the distributed workloads components).
- You have configured the distributed training resources as described in Managing distributed workloads.

Procedure

1. Log in to the OpenShift Console as a cluster administrator.
2. Enable NVIDIA GPU support in OpenShift AI.
   This process includes installing the Node Feature Discovery Operator and the NVIDIA GPU Operator. For more information, see Enabling NVIDIA GPUs.

   NOTE

   After the NVIDIA GPU Operator is installed, ensure that **rdma** is set to **enabled** in your **ClusterPolicy** custom resource instance.

3. To simplify the management of NVIDIA networking resources, install and configure the NVIDIA Network Operator, as follows:
   a. Install the NVIDIA Network Operator, as described in Adding Operators to a cluster in the OpenShift documentation.
   b. Configure the NVIDIA Network Operator, as described in the deployment examples in the Network Operator Application Notes in the NVIDIA documentation.
4. [Optional] To use Single Root I/O Virtualization (SR-IOV) deployment modes, complete the following steps:
   a. Install the SR-IOV Network Operator, as described in the Installing the SR-IOV Network Operator section in the OpenShift documentation.

b. Configure the SR-IOV Network Operator, as described in the Configuring the SR-IOV Network Operator section in the OpenShift documentation.

5. Use the Machine Configuration Operator to increase the limit of pinned memory for non-root users in the container engine (CRI-O) configuration, as follows:

a. In the OpenShift Console, in the **Administrator** perspective, click **Compute → MachineConfigs**.

b. Click **Create MachineConfig**.

c. Replace the placeholder text with the following content:

Example machine configuration

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 02-worker-container-runtime
spec:
  config:
    ignition:
      version: 3.2.0
    storage:
      files:
        - contents:
            inline: |
              [crio.runtime]
              default_ulimits = [
                "memlock=-1:-1"
              ]
          mode: 420
          overwrite: true
          path: /etc/crio/crio.conf.d/10-custom
```

d. Edit the **default_ulimits** entry to specify an appropriate value for your configuration. For more information about default limits, see the Set default ulimits on CRIO Using machine config Knowledgebase solution.

e. Click **Create**.

f. Restart the worker nodes to apply the machine configuration.

This configuration enables non-root users to run the training job with RDMA in the most restrictive OpenShift default security context.

Verification

1. Verify that the Operators are installed correctly, as follows:

a. In the OpenShift Console, in the **Administrator** perspective, click **Workloads → Pods**.

b. Select your project from the **Project** list.

c. Verify that a pod is running for each of the newly installed Operators.

2. Verify that RDMA is being used, as follows:

a. Edit the **PyTorchJob** resource to set the ***NCCL_DEBUG*** environment variable to **INFO**, as shown in the following example:

Setting the NCCL debug level to INFO

```
spec:
  containers:
  - command:
    - /bin/bash
    - -c
    - "your container command"
    env:
    - name: NCCL_SOCKET_IFNAME
      value: "net1"
    - name: NCCL_IB_HCA
      value: "mlx5_1"
    - name: NCCL_DEBUG
      value: "INFO"
```

b. Run the PyTorch job.

c. Check that the pod logs include an entry similar to the following text:

Example pod log entry

```
NCCL INFO NET/IB : Using [0]mlx5_1:1/RoCE [RO]
```

Additional resources

- Machine configuration in the OpenShift documentation
- Managing security context constraints in the OpenShift documentation

9.4. TROUBLESHOOTING COMMON PROBLEMS WITH DISTRIBUTED WORKLOADS FOR ADMINISTRATORS

If your users are experiencing errors in Red Hat OpenShift AI relating to distributed workloads, read this section to understand what could be causing the problem, and how to resolve the problem.

If the problem is not documented here or in the release notes, contact Red Hat Support.

9.4.1. A user's Ray cluster is in a suspended state

Problem

The resource quota specified in the cluster queue configuration might be insufficient, or the resource flavor might not yet be created.

Diagnosis

The user's Ray cluster head pod or worker pods remain in a suspended state. Check the status of the **Workload** resource that is created with the **RayCluster** resource. The **status.conditions.message** field provides the reason for the suspended state, as shown in the following example:

```
status:
 conditions:
   - lastTransitionTime: '2024-05-29T13:05:09Z'
     message: 'couldn''t assign flavors to pod set small-group-jobtest12: insufficient quota for
nvidia.com/gpu in flavor default-flavor in ClusterQueue'
```

Resolution

1. Check whether the resource flavor is created, as follows:
   a. In the OpenShift console, select the user's project from the **Project** list.
   b. Click **Home → Search**, and from the **Resources** list, select **ResourceFlavor**.
   c. If necessary, create the resource flavor.
2. Check the cluster queue configuration in the user's code, to ensure that the resources that they requested are within the limits defined for the project.
3. If necessary, increase the resource quota.

For information about configuring resource flavors and quotas, see Configuring quota management for distributed workloads.

9.4.2. A user's Ray cluster is in a failed state

Problem

The user might have insufficient resources.

Diagnosis

The user's Ray cluster head pod or worker pods are not running. When a Ray cluster is created, it initially enters a **failed** state. This failed state usually resolves after the reconciliation process completes and the Ray cluster pods are running.

Resolution

If the failed state persists, complete the following steps:

1. In the OpenShift console, select the user's project from the **Project** list.
2. Click **Workloads → Pods**.
3. Click the user's pod name to open the pod details page.
4. Click the **Events** tab, and review the pod events to identify the cause of the problem.
5. Check the status of the **Workload** resource that is created with the **RayCluster** resource. The **status.conditions.message** field provides the reason for the failed state.

9.4.3. A user's Ray cluster does not start

Problem

After the user runs the **cluster.apply()** command, when they run either the **cluster.details()** command or the **cluster.status()** command, the Ray cluster status remains as **Starting** instead of changing to **Ready**. No pods are created.

Diagnosis

Check the status of the **Workload** resource that is created with the **RayCluster** resource. The **status.conditions.message** field provides the reason for remaining in the **Starting** state. Similarly, check the **status.conditions.message** field for the **RayCluster** resource.

Resolution

1. In the OpenShift console, select the user's project from the **Project** list.
2. Click **Workloads → Pods**.
3. Verify that the KubeRay pod is running. If necessary, restart the KubeRay pod.
4. Review the logs for the KubeRay pod to identify errors.

9.4.4. A user cannot create a Ray cluster or submit jobs

Problem

After the user runs the **cluster.apply()** command, an error similar to the following text is shown:

```
RuntimeError: Failed to get RayCluster CustomResourceDefinition: (403)
Reason: Forbidden
HTTP response body: {"kind":"Status","apiVersion":"v1","metadata":
{},"status":"Failure","message":"rayclusters.ray.io is forbidden: User
\"system:serviceaccount:regularuser-project:regularuser-workbench\" cannot list resource
\"rayclusters\" in API group \"ray.io\" in the namespace \"regularuser-
project\"","reason":"Forbidden","details":{"group":"ray.io","kind":"rayclusters"},"code":403}
```

Diagnosis

The correct OpenShift login credentials are not specified in the **TokenAuthentication** section of the user's notebook code.

Resolution

1. Advise the user to identify and specify the correct OpenShift login credentials as follows:
   a. In the OpenShift console header, click your username and click **Copy login command**.
   b. In the new tab that opens, log in as the user whose credentials you want to use.
   c. Click **Display Token**.
   d. From the **Log in with this token** section, copy the **token** and **server** values.
   e. Specify the copied **token** and **server** values in your notebook code as follows:

```
auth = TokenAuthentication(
```

```
    token = "<token>",
    server = "<server>",
    skip_tls=False
)
auth.login()
```

2. Verify that the user has the correct permissions and is part of the **rhods-users** group.

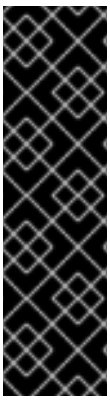
9.4.5. Additional resources

- Troubleshooting common problems with distributed workloads for users
- Troubleshooting common problems with Kueue

CHAPTER 10. CONFIGURING A CENTRAL AUTHENTICATION SERVICE FOR AN EXTERNAL OIDC IDENTITY PROVIDER

The built-in OpenShift OAuth server supports integration with various identity providers. However, it has limitations in direct OpenID Connect (OIDC) configurations on Red Hat OpenShift Service on AWS (ROSA) and on-premises OpenShift (OCP) 4.20 and later clusters. The internal **oauth** service can be disabled or removed, which breaks dependencies like **oauth-proxy** sidecar containers.

You can configure an external OIDC identity provider directly with Red Hat OpenShift AI by configuring a centralized Gateway API. The Gateway API configuration provides a secure, scalable, and manageable authentication solution because it centralizes the authentication logic and decouples it from individual backend services.



IMPORTANT

OpenID Connect (OIDC) configuration is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

10.1. ABOUT CENTRALIZED AUTHENTICATION GATEWAY API

A Gateway API with centralized authentication centralizes ingress traffic for all services behind a single domain, providing the following advanced capabilities:

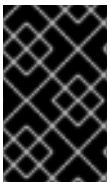
- Centralized authentication: A single authentication service requiring only one client ID and secret from the external OIDC Identity Provider (IDP).
- Simplified backend services: Backend services assume all incoming traffic is authenticated and contains necessary user headers.
- Authorization handling: Services still handle authorization at the service or pod level using sidecars like **kube-rbac-proxy**.
- Encrypted Communication: Traffic from the gateway to the backend services is fully encrypted with Transport Layer Security (TLS).

The Gateway API is implemented via an Istio Gateway on OpenShift (OCP) 4.19 and later. Since the Istio Gateway is built on the Envoy Proxy, it provides access to powerful Envoy-specific Custom Resource Definitions (CRDs), such as **EnvoyFilter**. The **opendatahub-operator** manages the deployment of **kube-auth-proxy**. The Operator then configures the Istio Gateway to use this service via an **EnvoyFilter** Custom Resource (CR).

For more information on supported OpenID Connect (OIDC) identity providers, see OpenShift documentation on Direct authentication identity providers

10.2. CONFIGURING OPENID CONNECT (OIDC) AUTHENTICATION FOR GATEWAY API

As a OpenShift AI administrator, you can configure an OpenID Connect (OIDC) authentication for Gateway API using parameters from your external OIDC identity provider.

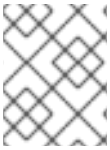


IMPORTANT

You must configure the OpenShift for direct authentication with an external OIDC identity provider before configuring the OpenShift AI Gateway for the Gateway to function properly.

Prerequisites

- You have configured the OpenShift for direct authentication with an external OIDC identity provider.
  - To configure OpenShift for direct authentication, follow the appropriate OpenShift documentation: Enabling direct authentication with an external OIDC identity provider .
  - To configure OpenShift for direct authentication using ROSA, follow the appropriate Red Hat OpenShift Service on AWS documentation: Creating an OpenID Connect configuration.



NOTE

You must configure OpenShift for direct authentication using the same OIDC provider that the Gateway uses.

- You have successfully installed and deployed OpenShift AI.
  - You have deployed the DataScienceCluster (DSC) and DSCInitialization. For more information, see Installing and deploying OpenShift AI .
  - You have deployed the OpenShift AI Operator in the **rhods-operator** namespace.
  - You have enabled Gateway API support on OCP 4.19 or later with Istio Gateway.
  - You have the following external authentication provider details:
    - Issuer URL
    - Client ID
    - Client Secret
    - Realm name (for Keycloak)
  - You have cluster administrator access which is required to create secrets and configure **GatewayConfig**.

For detailed step-by-step instructions, troubleshooting, and field definitions, refer to the OpenShift documentation on Configuring an external OIDC identity provider .

Procedure

1. In the OpenShift CLI (**oc**), verify the OpenShift authentication type by running the following command:
   -

```
$ oc get authentication.config/cluster -o jsonpath='{.spec.type}'
```

If the authentication is successful, you will see the following output: **OIDC**

2. Verify that your OIDC provider is configured as expected by running the following command:

```
$ oc get authentication.config/cluster -o jsonpath='{.spec.oidcProviders[*].name}'
```

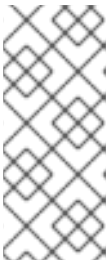
If the OIDC configuration is successful, you will see your provider name (e.g., **keycloak**).

3. Verify that the **kube-apiserver** has rolled out changes as expected.

```
$ oc get co kube-apiserver
```

If success is indicated, the expected output should look like the following example:

```
NAME             VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE
kube-apiserver   4.14.9    True        False         False      1d
```



NOTE

The rollout can take 20 minutes or more. Wait until all nodes have the new revision before proceeding. You can proceed to Gateway configuration steps when **oc get authentication.config/cluster** shows **type: OIDC**, **oc get co kube-apiserver** shows the authentication rollout is complete, and you can successfully authenticate to OpenShift using OIDC credentials.

4. Define the the following environment variables. You must replace the placeholder values with the actual details from your OIDC Identity Provider (IdP):

```
# Replace with your actual values
KEYCLOAK_DOMAIN="<keycloak.example.com>"
KEYCLOAK_REALM="<your-realm>"
KEYCLOAK_CLIENT_ID="<your-client-id>"
KEYCLOAK_CLIENT_SECRET="<your-client-secret>"
```

5. Create the client secret in the **openshift-ingress** namespace:

```
$ oc create secret generic keycloak-client-secret \
    --from-literal=clientSecret=$KEYCLOAK_CLIENT_SECRET \
    -n openshift-ingress
```

6. Update the **GatewayConfig** custom resource to enable OIDC authentication by patching it with the secret reference and OIDC details:

```
$ oc patch gatewayconfig default-gateway --type='merge' -p='{
  "spec": {
    "oidc": {
      "issuerURL": "https://'$KEYCLOAK_DOMAIN'/realms/'$KEYCLOAK_REALM'",
      "clientID": "'$KEYCLOAK_CLIENT_ID'",
      "clientSecretRef": {
        "name": "keycloak-client-secret",
        "key": "clientSecret"
```

```
        }
      }
    }
  }'
```

7. Verify that the client secret has been created and that the **GatewayConfig** shows the correct OIDC configuration:

```
$ oc get secret keycloak-client-secret -n openshift-ingress
$ oc get gatewayconfig default-gateway -o jsonpath='{.spec.oidc}'
```

Expected output for secret and **GatewayConfig** should look like the following example:

```
# Expected output (for secret)
NAME                   TYPE     DATA   AGE
keycloak-client-secret   Opaque   1      2m
# Expected output (for GatewayConfig)
{"clientID":"your-client-id","clientSecretRef":{"key":"clientSecret","name":"keycloak-client-secret"},"issuerURL":"https://keycloak.example.com/realms/your-realm"}
```

Verification

1. After configuring and authenticating the Gateway for your identity provider, you need to ensure that you can access your OpenShift console.
   a. Access the gateway by accessing the Console link:

```
$ oc get consolelink
```

   b. Login with your OIDC credentials and verify the following:
      i. You are redirected to the OIDC provider login page. A successful authentication redirects back to the Gateway.
      ii. Your OpenShift AI components are accessible (for example: Dashboard, Notebooks).
2. Check the **GatewayConfig** status to verify that the OIDC configuration was successfully provisioned:

```
$ oc get gatewayconfig default-gateway -o yaml
```

The expected output is the full YAML configuration of the **GatewayConfig** resource, showing the OIDC configuration details under **spec.oidc** and confirming successful deployment by displaying both the **Ready** and **ProvisioningSucceeded** conditions with a **status: "True"** value.

3. Verify the **kube-auth-proxy** deployment is running successfully in the **openshift-ingress** namespace:

```
$ oc get deployment kube-auth-proxy -n openshift-ingress
```

The expected output looks like the following example:

```
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
kube-auth-proxy   1/1     1            1           5m
```

4. Check the status and accessibility of the **data-science-gateway**:

```
$ oc get gateway data-science-gateway -n openshift-ingress
```

The expected output looks like the following example:

```
NAME                  CLASS                       ADDRESS
PROGRAMMED   AGE
data-science-gateway   data-science-gateway-class   aa87f5da7f0c748d5aa63b4916604108-
107643684.us-east-1.elb.amazonaws.com         True         5m
```

5. Test the OIDC discovery endpoint by running the following command:

```
curl -s https://your-keycloak-domain/realms/your-realm/.well-known/openid-configuration
```

The expected output is a JSON object containing the OIDC configuration endpoints (**issuer**, **authorization_endpoint**, **token_endpoint**, etc.) that confirm the OIDC provider is publicly discoverable.

Next steps

Once the external OIDC is configured and authenticated, the Cluster Administrator must perform the necessary authorization by mapping external Identity Provider (IdP) groups to specific OpenShift **ClusterRoles** to grant access to projects and resources.

1. Create a **ClusterRole** that grants users read and list access to OpenShift projects in the console:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: odh-projects-read
rules:
- apiGroups: ["project.openshift.io"]
  resources: ["projects"]
  verbs: ["get","list"]
```

2. Bind the **odh-projects-read** ClusterRole to your IdP group (for example, **odh-users**).

```
$ oc adm policy add-cluster-role-to-group odh-projects-read odh-users
```

3. Grant the ability to create and manage new projects by assigning the built-in self-provisioner ClusterRole to your group.

```
$ oc adm policy add-cluster-role-to-group self-provisioner odh-users
```

10.2.1. Security considerations

- Secret Management: Store OIDC client secrets securely and rotate them regularly.

- Network Policies: Consider implementing network policies to restrict access to the authentication proxy.
- TLS Configuration: Ensure all OIDC communication uses Transport Layer Security (TLS).
- Token Validation: While **kube-auth-proxy** validates tokens, ensure your OIDC provider is configured with appropriate token lifetimes.
- Audit Logging: Enable audit logging for authentication events.

10.3. TROUBLESHOOTING COMMON PROBLEMS WITH GATEWAY API CONFIGURATION

If your users are experiencing errors in Red Hat OpenShift AI relating to Gateway API configuration, read this section to understand what could be causing the problem, and how to resolve the problem.

If the problem is not documented here or in the release notes, contact Red Hat Support.

10.3.1. The GatewayConfig status shows as not ready

Problem

While setting up the OIDC, the **GatewayConfig** status shows as not ready. You see error messages about missing OIDC configuration and the **GatewayConfig** resource shows its status as **Ready: False**.

Diagnosis

1. Check **GatewayConfig** status by running the following command.

```
$ oc get gatewayconfig default-gateway -o yaml
```

2. Check for specific error messages by running the following command.

```
$ oc describe gatewayconfig default-gateway
```

The expected output confirms that the GatewayConfig resource is successfully provisioned by showing the OIDC configuration details under **Spec.Oidc** and displaying both the **Ready** and **ProvisioningSucceeded** status conditions with a **True** status.

3. Verify that the OIDC configuration is correct by running the following command.

```
$ oc get gatewayconfig default-gateway -o jsonpath='{.spec.oidc}'
```

Expected output shows the following example:

```
{"clientID":"your-client-id","clientSecretRef":{"key":"clientSecret","name":"keycloak-client-secret"},"issuerURL":"https://keycloak.example.com/realms/your-realm"}
```

Resolution

1. Verify the OIDC secret exists and is correct by running the following command.

```
$ oc get secret keycloak-client-secret -n openshift-ingress
```

2. Check OIDC issuer URL accessibility by running the following command.

```
curl -I https://your-keycloak-domain/realms/your-realm/.well-known/openid-configuration
```

The expected output confirms the OIDC issuer URL is accessible by returning the HTTP status code **HTTP/2 200** and the correct **content-type: application/json header**.

3. Ensure that the client Secret is correct.

10.3.2. Authentication proxy fails to start

Problem

The authentication proxy component fails to start after deploying **kube-auth-proxy**. The associated Pods are in a failing state, showing statuses such as **CrashLoopBackOff** or **Pending**, and the **kube-auth-proxy** Deployment is not ready.

Diagnosis

1. Check the **kube-auth-proxy** deployment status by running the following command.

```
$ oc get deployment kube-auth-proxy -n openshift-ingress
```

The expected output confirms that the deployment is successfully provisioned, showing **1/1** under the **READY** column.

2. Check the Pod logs by running the following command.

```
$ oc logs -l app=kube-auth-proxy -n openshift-ingress
```

The expected output confirms that the OAuth2 Proxy is configured and starting on the specified ports.

```
# Expected output
time="2024-01-15T10:30:00Z" level=info msg="OAuth2 Proxy configured"
time="2024-01-15T10:30:00Z" level=info msg="OAuth2 Proxy starting on :4180"
time="2024-01-15T10:30:00Z" level=info msg="OAuth2 Proxy starting on :8443"
```

3. Check the Pod events for errors by running the following command.

```
$ oc describe pod -l app=kube-auth-proxy -n openshift-ingress
```

The expected output should look like the following example.

```
# Expected output
Name:         kube-auth-proxy-7d4f8b9c6-xyz12
Namespace:    openshift-ingress
Status:       Running
Containers:
  kube-auth-proxy:
    State:          Running
    Ready:          True
```

```
    Restart Count:  0
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  5m    default-scheduler  Successfully assigned openshift-
ingress/kube-auth-proxy-7d4f8b9c6-xyz12 to worker-node-1
```

Resolution

1. Verify that the authentication secret contains the correct client secret by running the following command.

```
$ oc get secret kube-auth-proxy-creds -n openshift-ingress -o yaml
```

   The expected output should contain the keys **OAUTH2_PROXY_CLIENT_SECRET**, **OAUTH2_PROXY_COOKIE_SECRET**, and **OAUTH2_PROXY_CLIENT_ID**.

2. Check if the OIDC issuer URL is accessible from the cluster by running the following command.

```
curl -I https://your-keycloak-domain/realms/your-realm/.well-known/openid-configuration
```

   The expected output should return the HTTP status code **HTTP/2 200**.

3. Ensure that the client ID exists in your OIDC provider.

10.3.3. The Gateway is inaccessible

Problem

After configuring OIDC, you cannot access the Gateway URL: https://data-science-gateway.$CLUSTER_DOMAIN. Attempts to access the URL return 502 (Bad Gateway) or 503 (Service Unavailable) errors, indicating a networking failure that prevents external access or traffic routing to the service endpoint.

Diagnosis

1. Check the Gateway status of **data-science-gateway** by running the following command.

```
$ oc get gateway data-science-gateway -n openshift-ingress
```

   The expected output shows **PROGRAMMED** column as **True**, and a valid address is listed under the **ADDRESS** column.

2. Check the **HTTPRoute** status by running the following command.

```
$ oc get httproute -n openshift-ingress
```

   The expected output shows that the **oauth-callback-route** is present.

3. Check the **EnvoyFilter** by running the following command.

```
$ oc get envoyfilter -n openshift-ingress
```

   The expected output shows that the **authn-filter** is present.

4. Check the **kube-auth-proxy** Service by running the following command.

```
$ oc get service kube-auth-proxy -n openshift-ingress
```

The expected output shows that the Service and correct ports are present, like the following example:

```
# Expected output
NAME              TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)           AGE
kube-auth-proxy   ClusterIP   172.30.31.69   <none>        8443/TCP,9000/TCP   41h
```

Resolution

1. Verify the Gateway has a valid address by running the following command.

```
$ oc get gateway data-science-gateway -n openshift-ingress -o
jsonpath='{.status.addresses}'
```

The expected output shows a valid IP address or hostname.

2. Check if the **HTTPRoute** is properly configured by running the following command.

```
$ oc describe httproute oauth-callback-route -n openshift-ingress
```

The expected output confirms proper parent references and backend services.

3. Ensure the **EnvoyFilter** is applied correctly by running the following command.

```
$ oc describe envoyfilter authn-filter -n openshift-ingress
```

The expected outputconfirms the proper configuration for **kube-auth-proxy**.

10.3.4. The OIDC authentication fails

Problem

The OIDC authentication fails and you are unable to log in through the Gateway. You also experience symptoms such as redirect loops or explicit authentication errors after attempting to log in.

Diagnosis

1. Check the **kube-auth-proxy** logs for specific error messages by running the following command.

```
$ oc logs -l app=kube-auth-proxy -n openshift-ingress
```

The expected output confirms that the OAuth2 Proxy is configured and starting on the specified ports.

2. Verify the OIDC configuration in the **kube-auth-proxy** Secret by running the following command.

```
$ oc get secret kube-auth-proxy-creds -n openshift-ingress -o yaml
```

The expected output shows that the Secret contains the keys **OAUTH2_PROXY_CLIENT_ID**, **OAUTH2_PROXY_CLIENT_SECRET**, and **OAUTH2_PROXY_COOKIE_SECRET**. The output should look like the following example.

```
# Expected output
apiVersion: v1
kind: Secret
metadata:
  name: kube-auth-proxy-creds
  namespace: openshift-ingress
type: Opaque
data:
  OAUTH2_PROXY_CLIENT_ID: b2RoLWNsaWVudA==  # base64 encoded "data-science"
  OAUTH2_PROXY_CLIENT_SECRET: <base64-encoded-secret>
  OAUTH2_PROXY_COOKIE_SECRET: <base64-encoded-cookie-secret>
```

3. Test the OIDC discovery endpoint by running the following command.

```
curl -s https://your-keycloak-domain/realms/your-realm/.well-known/openid-configuration | jq .
```

The expected output returns the complete JSON configuration, including valid endpoints for **issuer**, **authorization_endpoint**, and **token_endpoint**.

Resolution

1. Log in to the OIDC provider (for example, Keycloak) and verify that the redirect URI registered for the client matches the expected endpoint on the Gateway: **https://data-science-gateway.$CLUSTER_DOMAIN/oauth2/callback**. Mismatches are a frequent cause of redirect loops.

2. Check if the client secret is properly set by running the following command.

```
echo $KEYCLOAK_CLIENT_SECRET | base64 -d
```

The expected output matches the secret in your OIDC provider.

3. Ensure that the issuer URL is accessible and correct by running the following command.

```
curl -I https://your-keycloak-domain/realms/your-realm/.well-known/openid-configuration
```

The expected output returns **HTTP/2 200**.

10.3.5. The dashboard is not accessible after authentication

Problem

After successfully authenticating with OIDC, you experience an authorization failure that prevents access to the dashboard. The failure results in 403 Forbidden errors while accessing the dashboard.

1. Check the **odh-dashboard** Deployment status by running the following command.

```
$ oc get deployment -n redhat-ods-applications rhods-dashboard
```

The expected outcome confirms that the Pods are running, similar to the following example.

```
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
rhods-dashboard   2/2     2            2           7h42m
```

2. Check the dashboard logs for any authorization errors by running the following command.

```
$ oc logs -l app=rhods-dashboard -n redhat-ods-applications
```

In the expected output, the logs confirm the Dashboard is running and ready to serve requests.

3. Verify the user permissions by running the following command.

```
$ oc auth can-i get projects --as=your-username
```

The expected output confirms that the user has the required access.

Resolution

1. Ensure that you have cluster-level RBAC permissions by running the following command.

```
$ oc adm policy add-cluster-role-to-user view your-username
```

The expected output confirms that the view cluster role has been added to the user.

2. Verify that the **odh-dashboard** HTTPRoute is properly configured with correct parent references (linking it to the Gateway) by running the following command.

```
$ oc get httproute rhods-dashboard -n redhat-ods-applications -o yaml
```

The expected output shows proper parent references to the Gateway.

3. Check if the user is in the expected groups that may have roles bound to them required by the dashboard.

```
$ oc get user your-username -o yaml
```

The expected output confirms that the user is in the **odh-users** group.

CHAPTER 11. BACKING UP DATA

Backing up OpenShift AI involves various components, including the OpenShift cluster and storage data.

11.1. BACKING UP STORAGE DATA

It is a best practice to back up the data on your persistent volume claims (PVCs) regularly.

Backing up your data is particularly important before you delete a user and before you uninstall OpenShift AI, as all PVCs are deleted when OpenShift AI is uninstalled.

For more information about backing up PVCs for your cluster platform, see OADP Application backup and restore in the OpenShift Container Platform documentation.

Additional resources

- Understanding persistent storage

11.2. BACKING UP YOUR CLUSTER

If you plan to upgrade or uninstall OpenShift AI on your cluster, back up your cluster data so that you can restore it later if needed.

For more information, see Backup and restore in the OpenShift Container Platform documentation.

CHAPTER 12. MANAGING OBSERVABILITY

Red Hat OpenShift AI provides centralized platform observability: an integrated, out-of-the-box solution for monitoring the health and performance of your OpenShift AI instance and user workloads.

This centralized solution includes a dedicated, pre-configured observability stack, featuring the OpenTelemetry Collector (OTC) for standardized data ingestion, Prometheus for metrics, and the Red Hat build of Tempo for distributed tracing. This architecture enables a common set of health metrics and alerts for OpenShift AI components and offers mechanisms to integrate with your existing external observability tools.

IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

12.1. ENABLING THE OBSERVABILITY STACK

The observability stack collects and correlates metrics, traces, and alerts for OpenShift AI so that you can monitor, troubleshoot, and optimize OpenShift AI components. A cluster administrator must explicitly enable this capability in the **DataScienceClusterInitialization** (DSCI) custom resource.

Once enabled, you can perform the following actions:

- Accelerate troubleshooting by viewing metrics, traces, and alerts for OpenShift AI components in one place.
- Maintain platform stability by monitoring health and resource usage and receiving alerts for critical issues.
- Integrate with existing tools by exporting telemetry to third-party observability solutions through the Red Hat build of OpenTelemetry.

IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

- You have installed Red Hat OpenShift AI.
- You have installed the following Operators, which provide the components of the observability stack:
  - **Cluster Observability Operator**: Deploys and manages Prometheus and Alertmanager for metrics and alerts.
  - **Tempo Operator**: Provides the Tempo backend for distributed tracing.
  - **Red Hat build of OpenTelemetry**: Deploys the OpenTelemetry Collector for collecting and exporting telemetry data.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the OpenShift console, click **Operators → Installed Operators**.
3. Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
4. Click the **DSCInitialization** tab.
5. Click the default instance name (for example, **default-dsci**) to open the instance details page.
6. Click the **YAML** tab to show the instance specifications.
7. In the **spec.monitoring** section, set the value of the **managementState** field to **Managed**, and configure metrics, alerting, and tracing settings as shown in the following example:

Example monitoring configuration

```
# ...
spec:
  monitoring:
    managementState: Managed              # Required: Enables and manages the observability stack
    namespace: redhat-ods-monitoring      # Required: Namespace where monitoring components are deployed
    alerting: {}                          # Alertmanager configuration, uses default settings if empty
    metrics:                              # Prometheus configuration for metrics collection
      replicas: 1                         # Optional: Number of Prometheus instances
      resources:                          # CPU and memory requests and limits for Prometheus pods
        cpulimit: 500m                    # Optional: Maximum CPU allocation in millicores
        cpurequest: 100m                  # Optional: Minimum CPU allocation in millicores
        memorylimit: 512Mi                # Optional: Maximum memory allocation in mebibytes
        memoryrequest: 256Mi              # Optional: Minimum memory allocation in mebibytes
      storage:                            # Storage configuration for metrics data
        size: 5Gi                         # Required: Storage size for Prometheus data
        retention: 90d                    # Required: Retention period for metrics data in days
      exporters: {}                       # External metrics exporters
    traces:                               # Tempo backend for distributed tracing
      sampleRatio: '0.1'                  # Optional: Portion of traces to sample, expressed as a decimal
      storage:                            # Storage configuration for trace data
```

```
      backend: pv                  # Required: Storage backend for Tempo traces (pv, s3, or
gcs)
      retention: 2160h             # Optional: Retention period for trace data in hours
    exporters: {}                  # External traces exporters
# ...
```

8. Click **Save** to apply your changes.

Verification

Verify that the observability stack components are running in the configured namespace:

1. In the OpenShift web console, click **Workloads → Pods**.
2. From the project list, select **redhat-ods-monitoring**.
3. Confirm that there are running pods for your configuration. The following pods indicate that the observability stack is active:

```
alertmanager-data-science-monitoringstack-#     2/2   Running   0   1m
data-science-collector-collector-#              1/1   Running   0   1m
prometheus-data-science-monitoringstack-#       2/2   Running   0   1m
tempo-data-science-tempomonolithic-#            1/1   Running   0   1m
thanos-querier-data-science-thanos-querier-#    2/2   Running   0   1m
```

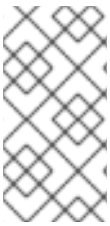
Next step

- *Collecting metrics from user workloads*

12.2. COLLECTING METRICS FROM USER WORKLOADS

After a cluster administrator enables the observability stack in your cluster, metric collection becomes available but is not automatically active for all deployed workloads. The monitoring system relies on a specific label to identify which pods Prometheus should scrape for metrics.

To include a workload, such as a user-created workbench, training job, or inference service, in the centralized observability stack, add the label **monitoring.opendatahub.io/scrape=true** to the pod template in the workload's deployment configuration. This ensures that all pods created by the deployment include the label and are automatically scraped by Prometheus.



NOTE

Apply the **monitoring.opendatahub.io/scrape=true** label only to workloads that expose metrics and that you want the observability stack to monitor. Do not add this label to operator-managed workloads, because the operator might overwrite or remove it during reconciliation.

Prerequisites

- A cluster administrator has enabled the observability stack as described in *Enabling the observability stack*.
- You have OpenShift AI administrator privileges or you are the project owner.

- You have deployed a workload that exposes a /**metrics** endpoint, such as a workbench server or model service pod.
- You have access to the project where the workload is running.

Procedure

1. Log in to the OpenShift web console as a cluster administrator or project owner.
2. Click **Workloads → Deployments**.
3. In the **Project** list at the top of the page, select the project where your workload is deployed.
4. Identify the deployment that you want to collect metrics from and click its name.
5. On the **Deployment details** page, click the **YAML** tab.
6. In the YAML editor, add the required label under the **spec.template.metadata.labels** section, as shown in the following example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <example_name>
  namespace: <example_namespace>
spec:
  template:
    metadata:
      labels:
        monitoring.opendatahub.io/scrape: 'true'
# ...
```

7. Click **Save**.
   OpenShift automatically rolls out a new ReplicaSet and pods with the updated label. When the new pods start, the observability stack begins scraping their metrics.

Verification

Verify that metrics are being collected by accessing the Prometheus instance deployed by OpenShift AI.

1. Access Prometheus by using a route:
   a. In the OpenShift web console, click **Networking → Routes**.
   b. From the project list, select **redhat-ods-monitoring**.
   c. Locate the route associated with the Prometheus service, such as **data-science-prometheus-route**.
   d. Click the **Location** URL to open the Prometheus web console.
2. Alternatively, access Prometheus locally by using port forwarding:
   a. List the Prometheus pods:

```
$ oc get pods -n redhat-ods-monitoring -l prometheus=data-science-monitoringstack
```

b. Start port forwarding:

```
$ oc port-forward __<prometheus-pod-name>__ 9090:9090 -n redhat-ods-monitoring
```

c. In a web browser, open the following URL:

```
http://localhost:9090
```

3. In the Prometheus web console, search for a metric exposed by your workload.
If the label is applied correctly and the workload exposes metrics, the metrics appear in the Prometheus instance deployed by OpenShift AI.

12.3. EXPORTING METRICS TO EXTERNAL OBSERVABILITY TOOLS

You can export OpenShift AI operational metrics to an external observability platform, such as Grafana, Prometheus, or any OpenTelemetry-compatible backend. This allows you to visualize and monitor OpenShift AI metrics alongside data from other systems in your existing observability environment.

Metrics export is configured in the **DataScienceClusterInitialization** (DSCI) custom resource by populating the **.spec.monitoring.metrics.exporters** field. When you define one or more exporters in this field, the OpenTelemetry Collector (OTC) deployed by OpenShift AI automatically updates its configuration to include each exporter in its metrics pipeline. If this field is empty or undefined, metrics are collected only by the in-cluster Prometheus instance that is deployed with OpenShift AI.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- The observability stack is enabled as described in *Enabling the observability stack*.
- The external observability platform can receive metrics through a supported export protocol.
- You know the URL of your external metrics receiver endpoint.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Click **Operators → Installed Operators**.
3. Select the **Red Hat OpenShift AI** Operator from the list.
4. Click the **DSCInitialization** tab.
5. Click the default DSCI instance, for example, **default-dsci**, to open its details page.
6. Click the **YAML** tab.
7. In the **spec.monitoring.metrics** section, add an **exporters** list that defines the external receiver configuration, as shown in the following example:

```
spec:
  monitoring:
    metrics:
      exporters:
```

```
- name: <external_exporter_name>
  type: <type>
  endpoint: https://example-otlp-receiver.example.com:4317
```

- **name**: A unique, descriptive name for the exporter configuration. Do not use reserved names such as **prometheus** or **otlp/tempo**.
- **type**: The protocol used for export, for example:
  - **otlp**: For OpenTelemetry-compatible backends using gRPC or HTTP.
  - **prometheusremotewrite**: For Prometheus-compatible systems that use the remote write protocol.
- **endpoint**: The full URL of your external metrics receiver. For OTLP, endpoints typically use port **4317** (gRPC) or **4318** (HTTP). For Prometheus remote write, endpoints typically end with **/api/v1/write**. For example:
  - **otlp**: **https://example-otlp-receiver.example.com:4317** (gRPC) or **https://example-otlp-receiver.example.com:4318** (HTTP)
  - **prometheusremotewrite**: **https://example-prometheus-remote.example.com/api/v1/write**

8. Click **Save**.
   The OpenTelemetry Collector automatically reloads its configuration and begins forwarding metrics to the specified external endpoint.

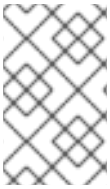
Verification

1. Verify that the OpenTelemetry Collector pods restart and apply the new configuration:

```
$ oc get pods -n redhat-ods-monitoring
```

   The **data-science-collector-collector-*** pods should restart and display a **Running** status.

2. In your external observability platform, verify that new metrics from OpenShift AI appear in the metrics list or dashboard.



NOTE

If you remove the **.spec.monitoring.metrics.exporters** configuration from the DSCI, the OpenTelemetry Collector automatically reverts to collecting metrics only for the in-cluster Prometheus instance.

12.4. VIEWING TRACES IN EXTERNAL TRACING PLATFORMS

When tracing is enabled in the **DataScienceClusterInitialization** (DSCI) custom resource, OpenShift AI deploys the Red Hat build of Tempo as the tracing backend and the Red Hat build of OpenTelemetry Collector (OTC) to receive and route trace data.

To view and analyze traces outside of OpenShift AI, complete the following tasks:

- Configure your instrumented applications to send traces to the OpenTelemetry Collector.

- Connect your preferred visualization tool, such as Grafana or Jaeger, to the Tempo Query API.

Prerequisites

- A cluster administrator has enabled tracing as part of the observability stack in the DSCI configuration.
- You have access to the monitoring namespace, for example **redhat-ods-monitoring**.
- You have network access or cluster administrator privileges to create a route or port forward from the cluster.
- Your application is instrumented with an OpenTelemetry SDK or library to generate and export trace data.

Procedure

1. Find the OpenTelemetry Collector endpoint.
   The OpenTelemetry Collector receives trace data from instrumented applications by using the OpenTelemetry Protocol (OTLP).
   a. In the OpenShift web console, navigate to **Networking → Services**.
   b. In the **Project** list, select the monitoring namespace, for example, **redhat-ods-monitoring**.
   c. Locate the Service named **data-science-collector** or a similar name associated with the OpenTelemetry Collector.
   d. Use the Service name or ClusterIP as the OTLP endpoint in your application configuration. Your application must export traces to one of the following ports on the collector service:
      - gRPC: **4317**
      - HTTP: **4318**
        Example environment variable:

        ```
        OTEL_EXPORTER_OTLP_ENDPOINT=http://data-science-collector.redhat-ods-monitoring.svc.cluster.local:4318
        ```

        NOTE

        See the Red Hat build of OpenTelemetry documentation for details about configuring application instrumentation.

2. Connect your visualization tool to the Tempo query service.
   You can use a visualization tool, such as Grafana or Jaeger, to query and display traces from the Red Hat build of Tempo deployed by OpenShift AI.
   a. In the OpenShift web console, navigate to **Networking → Services**.
   b. In the **Project** list, select the monitoring namespace, for example, **redhat-ods-monitoring**.
   c. Locate the Service named **tempo-query** or **tempo-query-frontend**.

d. To make the service accessible to external tools, a cluster administrator must perform one of the following actions:

- Create a route: Expose the Tempo Query service externally by creating an OpenShift route.
- Use port forwarding: Temporarily forward a local port to the Tempo Query service by using the OpenShift CLI (**oc**):

```
$ oc port-forward svc/tempo-query-frontend 3200:3200 -n redhat-ods-monitoring
```

After the port is forwarded, connect your visualization tool to the Tempo Query API endpoint, for example:

```
http://localhost:3200
```

NOTE

See the Tempo Operator documentation for details about connecting to Tempo.

Verification

1. Confirm that your instrumented application is generating and exporting trace data.
2. Verify that the OpenTelemetry Collector pod is running in the monitoring namespace:

```
$ oc get pods -n redhat-ods-monitoring | grep collector
```

The **data-science-collector-collector-*** pod should display a **Running** status.

3. Access your visualization tool and confirm that new traces appear in the trace list or search view.

12.5. ACCESSING BUILT-IN ALERTS

The centralized observability stack deploys a Prometheus Alertmanager instance that provides a common set of built-in alerts for OpenShift AI components. These alerts monitor critical platform conditions, such as operator downtime, crashlooping pods, and unresponsive services.

By default, the Alertmanager is internal to the cluster and is not exposed through a route. You can access the Alertmanager web interface locally by using the OpenShift CLI (**oc**).

Prerequisites

- You have OpenShift AI administrator privileges.
- The observability stack is enabled as described in *Enabling the observability stack*.
- You know the monitoring namespace, for example **redhat-ods-monitoring**.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform

- Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, log in to the OpenShift CLI (**oc**) as a cluster administrator:

```
$ oc login https://api.198.51.100.10:6443
```

2. Verify that the Alertmanager pods are running in the monitoring namespace:

```
$ oc get pods -n redhat-ods-monitoring | grep alertmanager
```

Example output:

```
alertmanager-data-science-monitoringstack-0   2/2   Running   0   2h
alertmanager-data-science-monitoringstack-1   2/2   Running   0   2h
```

3. Confirm that a ClusterIP service exposes the Alertmanager web interface on port 9093:

```
$ oc get svc -n redhat-ods-monitoring | grep alertmanager
```

Example output:

```
data-science-monitoringstack-alertmanager     ClusterIP   198.51.100.5   <none>   9093/TCP
```

4. Start a local port forward to the Alertmanager service:

```
$ oc port-forward svc/data-science-monitoringstack-alertmanager 9093:9093 -n redhat-ods-monitoring
```

5. In a web browser, open the following URL to access the Alertmanager web interface:

```
http://localhost:9093
```

Verification

- Confirm that the Alertmanager web interface opens at **http://localhost:9093** and displays active alerts for OpenShift AI components.

CHAPTER 13. VIEWING LOGS AND AUDIT RECORDS

As a cluster administrator, you can use the OpenShift AI Operator logger to monitor and troubleshoot issues. You can also use OpenShift audit records to review a history of changes made to the OpenShift AI Operator configuration.

13.1. CONFIGURING THE OPENSHIFT AI OPERATOR LOGGER

You can change the log level for OpenShift AI Operator components by setting the **.spec.devFlags.logmode** flag for the **DSC Initialization/DSCI** custom resource during runtime. If you do not set a **logmode** value, the logger uses the INFO log level by default.

The log level that you set with **.spec.devFlags.logmode** applies to all components, not just those in a **Managed** state.

The following table shows the available log levels:

Log level	Stacktrace level	Verbosity	Output	Timestamp type
devel or **development**	WARN	INFO	Console	Epoch timestamps
"" (or no **logmode** value set)	ERROR	INFO	JSON	Human-readable timestamps
prod or **production**	ERROR	INFO	JSON	Human-readable timestamps

Logs that are set to **devel** or **development** generate in a plain text console format. Logs that are set to **prod**, **production**, or which do not have a level set generate in a JSON format.

Prerequisites

- You have administrator access to the **DSCInitialization** resources in the OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. Log in to the OpenShift as a cluster administrator.
2. Click **Operators → Installed Operators** and then click the Red Hat OpenShift AI Operator.
3. Click the **DSC Initialization** tab.
4. Click the **default-dsci** object.

5. Click the **YAML** tab.
6. In the **spec** section, update the **.spec.devFlags.logmode** flag with the log level that you want to set.

```
apiVersion: dscinitialization.opendatahub.io/v1
kind: DSCInitialization
metadata:
  name: default-dsci
spec:
  devFlags:
    logmode: development
```

7. Click **Save**.

You can also configure the log level from the OpenShift CLI (**oc**) by using the following command with the **logmode** value set to the log level that you want.

```
oc patch dsci default-dsci -p '{"spec":{"devFlags":{"logmode":"development"}}}' --type=merge
```

Verification

- If you set the component log level to **devel** or **development**, logs generate more frequently and include logs at **WARN** level and above.
- If you set the component log level to **prod** or **production**, or do not set a log level, logs generate less frequently and include logs at **ERROR** level or above.

13.1.1. Viewing the OpenShift AI Operator logs

1. Log in to the OpenShift CLI (**oc**).
2. Run the following command to stream logs from all Operator pods:

```
for pod in $(oc get pods -l name=rhods-operator -n redhat-ods-operator -o name); do
  oc logs -f "$pod" -n redhat-ods-operator &
done
```

The Operator pod logs open in your terminal.

TIP

Press **Ctrl+C** to stop viewing. To fully stop all log streams, run **kill $(jobs -p)**.

You can also view each Operator pod log in the OpenShift console by navigating to **Workloads → Pods**, selecting the **redhat-ods-operator** project, clicking a pod name, and then clicking the **Logs** tab.

13.2. VIEWING AUDIT RECORDS

Cluster administrators can use OpenShift auditing to see changes made to the OpenShift AI Operator configuration by reviewing modifications to the DataScienceCluster (DSC) and DSCInitialization (DSCI) custom resources. Audit logging is enabled by default in standard OpenShift cluster configurations. For

more information, see Viewing audit logs in the OpenShift documentation.

NOTE

In Red Hat OpenShift Service on AWS, audit logging is disabled by default because the Elasticsearch log store does not provide secure storage for audit logs. To configure log forwarding, see Logging in the Red Hat OpenShift Service on AWS documentation.

The following example shows how to use the OpenShift audit logs to see the history of changes made (by users) to the DSC and DSCI custom resources.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. To access the full content of the changed custom resources, set the OpenShift audit log policy to **WriteRequestBodies** or a more comprehensive profile. For more information, see Configuring the audit log policy.
3. Fetch the audit log files that are available for the relevant control plane nodes. For example:

```
oc adm node-logs --role=master --path=kube-apiserver/ \
  | awk '{ print $1 }' | sort -u \
  | while read node ; do
      oc adm node-logs $node --path=kube-apiserver/audit.log < /dev/null
    done \
  | grep opendatahub > /tmp/kube-apiserver-audit-opendatahub.log
```

4. Search the files for the DSC and DSCI custom resources. For example:

```
jq 'select((.objectRef.apiGroup == "dscinitialization.opendatahub.io"
              or .objectRef.apiGroup == "datasciencecluster.opendatahub.io")
             and .user.username != "system:serviceaccount:redhat-ods-operator:redhat-ods-
operator-controller-manager"
             and .verb != "get" and .verb != "watch" and .verb != "list")' < /tmp/kube-apiserver-
audit-opendatahub.log
```

Verification

- The commands return relevant log entries.

TIP

To configure the log retention time, see the Logging section in the OpenShift documentation.